

Dynamic Orienteering on a Network of Queues

Shu Zhang

Jeffrey W. Ohlmann

Barrett W. Thomas

August 16, 2016

Abstract

We introduce a stochastic orienteering problem on a network of queues in which the traveler must arrive and enter service at locations within the respective time windows in order to collect rewards, but the traveler may experience random wait time at each location before service can begin. To maximize expected reward collected, the traveler must determine which locations to visit and how long to wait in queues at each location. We formally model the problem as a Markov decision process (MDP) with the objective of maximizing expected rewards. We investigate the existence of optimal control limits and examine conditions under which certain actions cannot be optimal. To solve the problem, we propose an approximate dynamic programming approach based on rollout algorithms. The method introduces a two-stage heuristic estimation that we refer to as compound rollout. In the first stage, the algorithm decides whether to stay at the current location or go to another location. If departing the current location, it chooses the next location in the second stage. We demonstrate the value of our modeling and solution approaches by comparing the dynamic policies to a-priori-route solutions with recourse actions.

1 Introduction

The orienteering problem (OP) is a routing problem in which given a set of locations, each associated with a reward, the traveler needs to develop a path to visit a subset of the locations with an objective to maximize the total reward collected. The OP has been widely studied in literature since 1980s (see Vansteenwegen et al. (2011) for a review). It has logistics applications wherever deliveries or services have to be made at customers' locations to receive payment, and only a subset of customers' requests can be satisfied due to limited business hours (Campbell et al. (2011)). In this study, we introduce a variant to the orienteering problem, named as the orienteering on a network

of queues, where the traveler may find that there is a queue of others waiting at the location she visits. To collect reward, the traveler must wait in the queue. The queue length is unknown to the traveler before her arrival, and the wait time is random while the traveler is in a queue. This problem is applicable to several operations, such as routing taxis to taxi stands in metropolitan area where there may be queues of competitors (Cheng and Qu (2009)), designing trips for tourists who may experience queueing at attractions (Vansteenwegen and Souffriau (2010)), and scheduling a salesperson to visit customers where the salesperson may experience random wait time due to queues of other individuals wishing to meet with the customer (such as in pharmaceutical and medical device sales, Zhang et al. (2014)).

The queue at each location consists of the traveler’s competitors. The wait time in queue is random due to the randomness in the arrival and departure of competitors and the time they spend in waiting. Given a list of locations from whom the traveler can collect reward, she needs to decide which locations to visit and the order to visit them to maximize the total expected reward. Upon arrival, the traveler needs to decide whether to join the queue and wait or to depart immediately to visit another location. Deciding to queue at a location, the traveler must periodically determine whether to renege and go to another location. When choosing whom to visit next, the traveler considers locations whom she has not visited and locations she visited but has not entered service and collected a reward. We consider that the reward with each location can only be collected if the traveler enters service at the location within the location’s pre-determined time window. We refer to this problem as the dynamic orienteering problem on a network of queues with time windows (DOPTW).

We formally model the problem as a Markov decision process (MDP). To overcome significant runtime challenges, we develop an approximate dynamic programming approach based on rollout algorithms to solve the problem. The value of the decision of whether to stay and where to go is evaluated hierarchically by different approximations. We refer to this approach as *compound rollout*. In compound rollout, decisions are made in two stages. In the first stage, the compound rollout decides whether to stay at the current location or go to another. If the first-stage decision is to go, in the second stage it decides to which location to go.

This paper makes the following contributions to the literature. First, we introduce a new dynamic orienteering problem in which a traveler may experience random wait times upon visiting

locations. Second, we identify conditions under which certain actions can be eliminated from the action set of a given state. Third, we investigate the existence of optimal control limits and identify the limited existence of optimal control limit policies. Fourth, we propose a novel compound rollout heuristic to facilitate decision making. The compound rollout is distinct in implementing different mechanisms for reward-to-go approximation based on explicitly partitioning the action space. Finally, our computational results demonstrate the capability of compound rollout in making high quality decisions with reasonable computational efforts by comparing our dynamic policies to benchmark a priori routing solutions.

In §2, we review related literature to position our contribution. In §3, we provide a MDP model for the problem and present the derivation of state transition probabilities and action elimination conditions. We investigate the existence of optimal control limit policies and present the results in the electronic companion. We describe our rollout algorithms in §4 and present computational results of our rollout methodology in §5. In §6, we conclude the paper and discuss future work.

2 Literature Review

The DOPTW is a variant of the stochastic traveling salesman problem (TSP) with profits and time constraints. The most closely related literature to the work in this paper is the study by Zhang et al. (2014) on an orienteering problem with time windows and stochastic wait times at locations. Their problem is motivated by the decision-making of pharmaceutical representatives who visit doctors to build brand awareness. They consider random wait times at customers resulting from queues of competing sales representatives and develop a priori solution with recourse actions. Similar to Zhang et al. (2014), in the DOPTW, the traveler faces a network of queues, but rather than implementing static decision rules in a priori routing, we propose dynamic policies that enable the traveler to make the routing decision as well as the queueing decision based on realized information at each decision epoch. We note that such dynamic policies allow revisiting a location, which is not possible in the a priori solutions.

The problem studied in this paper and in Zhang et al. (2014) can be seen as a generalization of the problem presented in Gupta et al. (2014). Like this paper, Gupta et al. (2014) consider stochastic orienteering problem in which the traveler may need to wait before collecting the reward.

Unlike in this paper, however, Gupta et al. (2014) does not address balking and reneging and does not explicitly model queueing. Further, the paper also does not consider customer time windows and does not allow customers to be revisited. Gupta et al. (2014) present approximation algorithms for the a priori and dynamic cases.

Other recent studies on stochastic TSP with profits and time constraints focus on developing a priori solutions. Examples include Teng et al. (2004), Tang and Miller-Hooks (2005), Campbell et al. (2011), Papapanagiotou et al. (2013), and Evers et al. (2014). In the above studies, hard time windows are not considered and the random arrival times result from the stochastic travel and service times. In Voccia et al. (2013), hard time windows are considered in a probabilistic traveling salesman problem (PTSPTW), but the objective of the PTSPTW is to minimize the expected traveling costs, and the random arrival times are induced via random presence of customers. In the DOPTW, there is a hard time window associated with each location and the randomness in arrival time is induced by the stochastic waiting and service times at previous locations.

While literature on the stochastic TSP with profits and time constraints mainly focuses on developing a priori routes and we know of none that incorporate rollout algorithms, dynamic solutions have been examined for vehicle routing problems (VRPs). Our work is similar methodologically to approximate dynamic programming (ADP) approaches applied to dynamic routing problems, particularly methods that develop heuristic routing policies via rollout procedures (Secomandi, 2000, 2001; Novoa and Storer, 2009; Goodson et al., 2013; Toriello et al., 2014; Goodson et al., 2015; Klapp et al., to appear; Ulmer et al., 2015a,b). The compound rollout algorithm we propose for the DOPTW involves executing rollout policies as in the above studies. However, our study is distinct in explicitly partitioning the action space and implementing different mechanisms to approximate reward-to-go based on the partition.

Our work is also related to the queueing literature investigating the decision rules in queueing systems. Yechiali (1971) and Yechiali (1972) investigate the optimal balking/joining rules in a $G1/M/1$ and a $G1/M/s$ system, respectively. They demonstrate that for both systems, a non-randomized control limit optimizing a customer's long-run average reward exists. Burnetas (2013) examine a system consisting of a series of $M/M/m$ queues, in which a customer receives a reward by completing service at a queue and incurs a delay cost per unit of time in queue. The customer needs to decide whether to balk or to enter a queue, with an objective to maximize the total net benefit.

Once balking, the customer leaves the system permanently. Honnappa and Jain (2015) study the arrival process in a network of queues, where travelers choose when to arrive at a parallel queuing network and which queue to join upon arrival. They focus on deriving the equilibrium arrival and routing profiles. Similar to the above studies, we investigate whether or not the traveler should join the queue when arriving at a location. However, in the DOPTW, the traveler is routing through a network of queues, where queueing decisions of balking and renegeing are made and revisiting queues is allowed.

We note that in the wireless communication literature, studies have addressed the management of queues at servers, such as Kim et al. (2013), Yaghmaee et al. (2013), and Manfredi (2014). Kim et al. (2013) investigate admission control policies in mobile hotspots. They develop an optimization model for determining the queue size with an objective to maximize channel utilization. Yaghmaee et al. (2013) focus on developing a congestion control protocol for wireless body sensor networks. They propose a queue management approach such that data transmission rate is adjusted according to the priority of nodes in the network. Manfredi (2014) propose a cooperation based rate control scheme in the control of communication networks. The control scheme stabilizes and balances the queueing networks, reduces data packet loss, and improves link utilizations. Though the above studies investigate the control of a queueing network, they do not address the decision making faced by a routing entities as we do. As far as we are aware, this paper is the only study in literature that considers the queueing process in a dynamic routing problem.

3 Problem Statement

We formulate the DOPTW as a Markov decision process (MDP). In §3.1, we present the dynamic programming formulation. In §3.2, we present the dynamics of state transition. In §3.3, we investigate conditions under which actions that are guaranteed to not be optimal can be eliminated.

3.1 Problem Formulation

We define the problem on a complete graph $G = (V, E)$. The set V of n nodes corresponds to n potential locations that a traveler may visit. The set E consists of edges associated with each pair of vertices. We assume a deterministic travel time on edge (i, j) , denoted as c_{ij} . We set $c_{ij} = 0$ for $i = j$

and assume the triangle inequality holds, i.e., $c_{li} + c_{ij} < c_{lj}$ for all $i, j, l \in V$. A time window $[e_i, l_i]$ is associated with each location $i \in V$. In this study, we assume that the traveler can collect reward at location i before or at l_i but not before e_i . We assume the traveler departs as early as necessary to arrive at the first location in her tour before the time window begins. We use r_i to represent the reward associated with location i . Let M_a be the random variable with a known distribution representing the service time at location a , and let this distribution be bounded by a finite minimum value $\tilde{m}_a$ and a finite maximum value $\hat{m}_a$. Let Ψ_i be the random variable representing the traveler's arrival time at location i and ψ_i be a realization of Ψ_i . Queues of competitors may form at each location over time. We assume that there is a maximum possible queue length, denoted by L . The evolution of the queue length is governed by competitor arrivals and competitor departures upon completing reward collection at each location. Let X_i be the random variable representing the competitors' inter-arrival time at location i and x_i be a realization of X_i . Let Y_i be the random variable representing the service time of a competitor at location i and y_i be a realization of Y_i . Due to the random arrivals and departures of the traveler's competitors, the wait time in queue for the traveler is random.

We consider a discrete-time planning horizon $0, 1, \dots, T$, where $T = \max_{i \in V} l_i$ is the time after which no more decisions are made. While, in general, the traveler may execute a decision at each time increment, we define particular points in time in which decision epochs occur. Technically speaking, the resulting system dynamics define a semi-Markov decision process. Let Ξ_k be a random variable representing the time of the k th decision epoch and ξ_k be the realization of Ξ_k . A decision epoch is triggered by one of the following conditions:

- if the traveler is en route towards location i , the next decision epoch is triggered by the arrival of the traveler, i.e., $\xi_k = \psi_i$;
- if the traveler is waiting at a location, the next decision epoch is triggered by observing the first queueing event(s) or reaching a specified amount of time δ_i , whichever comes first, where δ_i is the longest time that the traveler will wait before making a decision while at location i . Thus, $\xi_k = \xi_{k-1} + \min\{x_i, y_i, \delta_i\}$. We note that a departure or a simultaneous arrival and departure will trigger a decision epoch because the time has advanced.

The state of the system represents the sufficient information for the traveler to execute a decision

of whether to stay and wait at the current location or to leave and go to another location. The state consists of information on the traveler's status as well as the status of each location. The status of the traveler is captured by the triple (t, d, q) , where $q \in [0, L]$ is the queue length at the location $d \in V$ at the current system time t .

We represent the status of locations by partitioning V into three sets, H , U , and W . The set $H \subseteq V$ represents the set of locations that the traveler has met, thereby collecting a reward. The set $U \subseteq V$ represents the set of locations whom the traveler has not yet visited. The set $W \subseteq V$ represents the set of locations that the traveler has visited but departed from before entering service and collecting reward. For each location $w \in W$, the state includes information $(\check{t}_w, \check{q}_w)$ representing the queue length $\check{q}_w$ observed at the time $\check{t}_w$ that the traveler departed location w . The traveler can then use this information to evaluate a decision to revisit a location $w \in W$. Let $(\check{t}, \check{q}) = (\check{t}_w, \check{q}_w)_{w \in W}$ denote the vector of information regarding the time and queue length at the locations the traveler has previously visited but did not enter service.

Thus, we represent the complete state of the system with the tuple $(t, d, q, H, U, W, (\check{t}, \check{q}))$ consisting of the information on the traveler's status as well as the status of the set of locations. The state space S is defined on $[0, T] \times V \times [0, L] \times V \times V \times V \times \{([0, T], [0, L])\}^V$. The initial state is $s_0 = (0, 0, 0, \emptyset, V, \emptyset, (\emptyset, \emptyset))$. An absorbing state must meet one of the following conditions:

- (i) the traveler has collected reward from all locations;
- (ii) it is infeasible for the traveler to arrive at any location in $U \cup W$ within the location's time window.

Thus, the set of absorbing states is defined as $S_K = \left\{ (t, d, q, H, U, W, (\check{t}, \check{q})) : H = V \text{ or } t + c_{di} \geq l_i, \forall i \in \{V \setminus H\} \right\}$.

At each decision epoch, the traveler selects an action that determines the traveler's location at the next decision epoch. At a location d at time t , a traveler can either stay at location d if she has not yet collected reward from location d or go to another location $i \in U \cup W$ with $l_i \geq t + c_{di}$. Thus, the action space for state s is $A(s) = \left\{ a \in V : t + c_{da} \leq l_a, a \in \{U \cup W\} \right\}$.

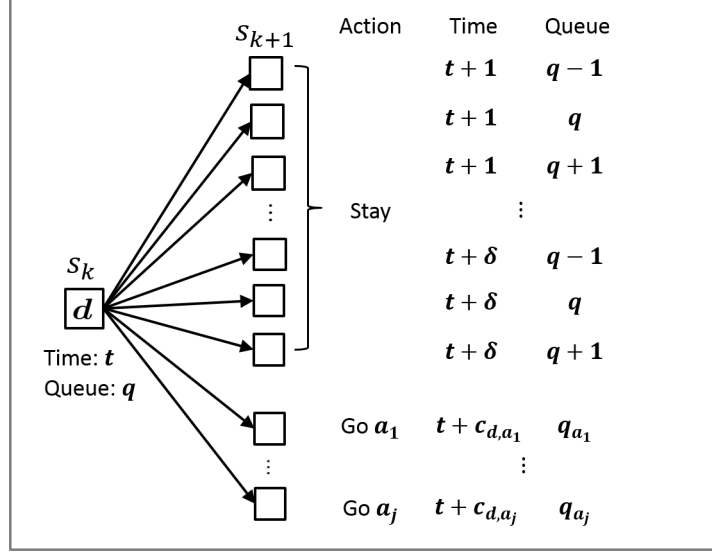


Figure 1: State Transition

3.2 System Dynamics

Figure 1 depicts the state transition that occurs upon action selection. In the remainder of this section, we describe this state transition in more detail. At decision epoch k with state $s_k = (t, d, q, H, U, W, (\check{t}, \check{q}))$, an action $a \in A(s_k)$ is selected, initiating a transition from state s_k to state $s_{k+1} = (t', d', q', H', U', W', (\check{t}', \check{q}'))$. The location at s_{k+1} is the selected action, $d' = a$. Let $P\{s_{k+1}|s_k, a\}$ be the transition probability from state s_k to s_{k+1} when selecting action a in s_k . At each epoch, the transition probability is defined by one of two action cases: (i) the traveler decides to depart from the current location and go to another location, or (ii) the traveler decides to stay at the current location and wait in line.

If the selected action in state s_k is to go to another location a , $P\{s_{k+1}|s_k, a\}$ is specified by the queue length distribution at location a at the arrival time, $t + c_{da}$. If the observed queue length is $q' > 0$ when the traveler arrives at location a , then the traveler is unable to immediately collect reward at time $t' = t + c_{da}$. If the observed queue length is $q' = 0$, the traveler collects reward r_a . Thus, when the selected action is to go to location a , we update the current time as

$$t' = \begin{cases} t + c_{da}, & \text{if } q' > 0, \\ t + c_{da} + m_a, & \text{if } q' = 0, \end{cases}$$

where m_a is a realization of M_a .

In the second case, if the selected action in state s_k is to stay at the current location d , the time

of the next state s_{k+1} , $t + \min\{X_d, Y_d, \delta_i\}$, is either the time of the first queueing event(s) or that of reaching the specified decision-making interval δ_i . The impact of a queueing event, particularly the arrival of a competitor, depends on the queueing discipline. The choice of queueing discipline affects the dynamics and transition probabilities. However, our model can be easily adapted for any choice, and our solution approach is amenable to any chosen queueing discipline. To present a concrete model and to offer a precise description of the instances used in the computational study, we focus our model here on priority queues at each location. Particularly, we assume that the traveler has the lowest priority in the queue. That is, when seeking to collect reward at a location, the traveler must wait as long as there are others in the queue, regardless of whether the competitors arrive earlier or later than the traveler. The traveler's competitors in the queue will be served via a first-come, first-served rule.

Given the priority queueing discipline described above, the first queueing event may be a competitor arrival that increases the queue length by one, a competitor departure that decreases the queue length by one, or both an arrival and a departure where the queue length remains the same as in state s_k . If no queueing events occur before or during the decision-making interval $[t, t + \delta_i]$, the queue length remains the same. If $q' = 0$ at time $t + \min\{X_d, Y_d, \delta_i\}$, the traveler collects reward r_d from location d and spends m_d time units of service time. Otherwise, no reward is collected. Finally, when the selected action is to stay at the current location, the current time is updated as

$$t' = \begin{cases} t + \min\{X_d, Y_d, \delta_i\}, & q' > 0, \\ t + \min\{X_d, Y_d, \delta_i\} + m_d, & q' = 0. \end{cases}$$

Regardless of whether the selected action is “to go” or “to stay,” we update the set of locations from which rewards have been collected by

$$H' = \begin{cases} H, & \text{if } q' > 0, \\ H \cup \{a\}, & \text{if } q' = 0. \end{cases}$$

If $q' > 0$, the set of locations from which rewards have been collected remain the same as in state s_k . If $q' = 0$, location a is added to set H . We update the set of unvisited locations by

$$U' = \begin{cases} U, & \text{if } a = d, \\ U \setminus \{a\}, & \text{if } a \neq d. \end{cases}$$

If $a = d$, the set of unvisited locations remains the same as in state s_k . If $a \neq d$ and a was previously unvisited, location a is removed from the set U . We update the set of locations that the traveler has visited but has not entered service by

$$W' = \begin{cases} W, & \text{if } a = d, q' > 0 \text{ or } a \neq d, q' = 0, \\ W \setminus \{a\}, & \text{if } a = d, q' = 0, \\ W \cup \{a\}, & \text{if } a \neq d, q' > 0. \end{cases}$$

If the traveler stays at the current location and does not collect reward ($a = d, q' > 0$), or if the traveler goes to another location a and collects reward ($a \neq d, q' = 0$), the set of locations that the traveler has visited but has not entered service remains the same as in s_k . If the traveler stays and collects reward from the current location ($a = d, q' = 0$), this location is removed from set W . If the traveler goes to another location a and observes a queue ($a \neq d, q' > 0$), location a is added to set W . Finally, we update information $(\check{t}', \check{q}')$ by adding $(t_a, q_a) = (t', q')$ to $(\check{t}, \check{q})$ if $q' > 0$ and by removing (t_a, q_a) from $(\check{t}, \check{q})$ if $q' = 0$.

Let Π be the set of all Markovian deterministic policies for the problem. A policy $\pi \in \Pi$ is a sequence of decision rules: $\pi = (\rho_0^\pi(s_0), \rho_1^\pi(s_1), \dots, \rho_K^\pi(s_K))$, where $\rho_k^\pi(s_k) : s_k \mapsto A(s_k)$ is a function specifying the action to select at decision epoch k while following policy π and K is the final decision epoch. For notational simplicity, we denote $\rho_k^\pi(s_k)$ using ρ_k^π . We seek a policy $\pi \in \Pi$ that maximizes the total expected reward, conditional on initial state s_0 : $E \left[\sum_{k=1}^K R_k(s_k, \rho_k^\pi(s_k)) | s_0 \right]$, where $R_k(s_k, a)$ is the expected reward collected when action a is selected in state s_k . Let $V_k(s_k)$ be the expected reward-to-go from state s_k in decision epoch k through K . Then an optimal policy can be obtained by solving $V_k(s_k) = \max_{a \in A(s_k)} \{R_k(s_k, a) + E[V_{k+1}(s_{k+1}) | s_k, a]\}$ for each decision epoch k and corresponding state s_k . In the following discussion, for notational simplicity, we denote $V_k(s_k)$ via $V(s_k)$.

3.3 Structural Results

For many decision-making problems under uncertainty, there exist optimal control-limit or threshold-type policies. Thus, it is natural to investigate whether there exists a threshold queueing policy in our problem. While we are unable to prove results for the general problem, we prove that at any given time, there exists a control limit in queue length under the assumption that the sequence of visiting customers is fixed. That is, there exists a threshold queue length such that if it is optimal

to stay at a location when observing this queue length, it is also optimal to stay when observing a shorter queue at the same time. We note that while we can show the existence of such policies, it is not obvious how to compute the threshold. Further, in our case, the result has limited applicability because we explore dynamic solutions to the problem and the assumption of a fixed visiting sequence cannot be held. We present the detailed discussion of this issue in § C.

In the remainder of this section, we investigate conditions under which actions are guaranteed to be suboptimal. By eliminating the suboptimal actions, we are able to reduce action space and improve the computational efficiency of our approximate dynamic programming approach that will be discussed in the next section. Let $o_i(\leq e_i)$ be the earliest time that the traveler's competitors start to queue at location i . We assume that the traveler is aware of the value of o_i . The following results hold regardless of queueing discipline. We provide proofs in the electronic companion.

Theorem 1. *If the traveler is in a queue of length $q > 0$ at location i at time $t < o_j - c_{ij} - \hat{m}_i - 1$, then the action of leaving location i to go to location j can be eliminated at time t .*

Theorem 2. *Assuming that the minimum service time is one time unit and that competitors in the queue will not renege, if the traveler is in a queue of length $q > 0$ at location i at time t and $t + q > l_i$, then the action of staying at location i can be eliminated.*

We note that Theorem 1 and Theorem 2 are applicable when the traveler is making staying decisions. If using the one-step rollout algorithm presented in §4.2 to selection an action, we need to evaluate the “staying” action together with all the “going” actions. As discussed in the next section, we need to develop heuristic policies to evaluate each candidate action and execute heuristic search procedures to find the heuristic policies. Thus, by eliminating actions via Theorem 1 and Theorem 2, we can reduce the computational efforts in constructing heuristic policies. We note that the reduction of computational complexity depends on information on customers, such as time windows, the earliest time that the traveler's competitors start to queue at each customer, the service time at each customer, and the traveling time between customers. In §5.3, we present the number of actions eliminated via Theorem 1 and Theorem 2 while solving 10-customer instances using the one-step rollout algorithm.

4 Rollout Policies

To find an optimal policy for the traveler, we must solve the optimality equation $V(s_k) = \max_{a \in A(s_k)} \{R_k(s_k, a) + E[V(s_{k+1})|s_k, a]\}$ for state s_k at each decision epoch k . However, given the curse of dimensionality present in the model and the limits of our structural results, it is not practical to exactly determine optimal policies. Instead, we turn to rollout algorithms to develop rollout policies. A form of approximate dynamic programming (see Powell (2007) for a general introduction to approximate dynamic programming), rollout algorithms construct rollout policies by employing a forward dynamic programming approach and iteratively using heuristic policies to approximate the reward-to-go at each decision epoch. Specifically, from a current state s_k at decision epoch k , rollout algorithms select an action a based on $\hat{V}(s_k) = \max_{a \in A(s_k)} \{R_k(s_k, a) + E[\hat{V}(s_{k+1})|s_k, a]\}$, where $\hat{V}(s_{k+1})$ is approximated by the value of heuristic policies. For an in-depth discussion on rollout algorithms, we refer the readers to Bertsekas et al. (1997), Bertsekas (2005), and Goodson et al. (2016).

As discussed in §3.2, the proposed solution approach does not require any particular queueing discipline. The approach also does not rely on any particular form of the transition probabilities and is amenable to heterogeneous inter-arrival and service times, even if our computational experiments assume homogeneity.

In §4.1, we present a heuristic a-priori-route policy for estimating the reward-to-go. In §4.2, we briefly summarize existing rollout algorithm methodology from the literature. In §4.3, we propose a compound rollout algorithm that is based on a partitioned action space and hierarchical application of heuristic policies.

4.1 A-Priori-Route Policy

To estimate the reward-to-go at a decision epoch, a rollout algorithm requires a heuristic policy to apply along the possible sample paths, where a heuristic policy is a suboptimal policy for all future states. For the DOPTW, we use a class of *a-priori-route heuristic policies* to approximate the reward-to-go. We choose a priori policies for this purpose for two reasons. One, a priori route policies are commonly used as solutions for dynamic routing problems (Campbell and Thomas, 2008). Second, a priori route policies have been shown to be more effective than other heuristics for

rollout algorithms applied to routing problems with stochastic demand (an analog to the stochastic rewards in this paper) (Novoa and Storer, 2009). Other examples of a priori policies being used in conjunction with rollout algorithms can be found in Goodson et al. (2013), Goodson et al. (2015), and Klapp et al. (to appear).

Given a state s_k , the corresponding *a-priori-route policy* $\pi(\nu)$ is characterized by an a priori route $\nu = (\nu_1, \nu_2, \dots, \nu_m)$, which specifies a pre-planned order of visits to m locations for the traveler (see Campbell and Thomas (2008) for an overview on a priori routing). The a priori route starts at the current location d at time t of state s_k (i.e., $\nu_1 = d$), followed by a sequence of $m - 1$ locations in set $\{\{U \cup W\} \setminus \{d\}\}$. The realized queue information at the current location d is given by the value of q in state s_k .

To account for the random information realized during the execution of an a priori route, we implement the two types of recourse actions proposed by Zhang et al. (2014). The first recourse action, skipping the next location in the sequence, is motivated by the notion that if the traveler arrives late in the time window of a location, she may be unlikely to collect reward due to the length of the queue. Zhang et al. (2014) establish a static rule stating that the traveler will skip location i if the traveler cannot arrive by a specified time $\tau_i \leq l_i$. The second recourse action corresponds to the queueing behaviors of balking and reneging. Zhang et al. (2014) utilize the queue length observed upon arrival to establish a static rule setting the longest amount of time γ_i that the traveler will wait at location i . With the wait time distributions presented in the electronic companion, we implement the static decision rules established by Zhang et al. (2014) to determine the value of τ_i and γ_i for each location i in an a priori route.

We execute a variable neighborhood descent (VND) procedure to search for an a-priori-route from the space of all possible a priori routes associated with state s . We outline the VND algorithm in §D of the electronic companion. We denote by $V^{\pi(\nu)}(s)$ the expected value of following a-priori-route policy $\pi(\nu)$ from state s . The objective is to find an a priori route ν^* that induces the optimal a-priori-route policy $\pi(\nu^*)$ such that $V^{\pi(\nu^*)}(s) > V^{\pi(\nu)}(s)$ for every $\pi(\nu)$. To reduce the computational burden of exactly evaluating the objective of $V^{\pi(\nu)}(s)$, we use Monte Carlo simulation to estimate $V^{\pi(\nu)}(s)$ collected from every neighbor policy.

4.2 Rollout Algorithms

In general, a rollout algorithm develops rollout policies by looking ahead at each decision epoch and approximating reward-to-go through heuristic policies, such as the a-priori-route policy presented in §4.1. Depending on how many steps the rollout procedure looks ahead before applying the heuristic, rollout algorithms can be categorized as one-step or multi-step rollout (Bertsekas (2005), Secomandi (2001), Novoa and Storer (2009)). More recently, Goodson et al. (2016) characterize pre- and post-decision rollout which can be viewed as zero-step and half-step look-ahead procedures based on the pre-decision and post-decision states of stochastic dynamic programming. In the remainder of this section, we focus on describing one-step and pre-decision rollout decision rules, which we incorporate in our compound rollout algorithm.

When occupying state s_k and evaluating action a , the one-step rollout transitions to all possible states at the next decision epoch $s_{k+1} \in S(s_k, a)$, where $S(s_k, a) = \{s_{k+1} : P\{s_{k+1}|s_k, a\} > 0\}$. For each of these possible future states s_{k+1} , we obtain an a-priori-route policy $\pi(\nu, s_{k+1})$ by executing the VND heuristic presented in the electronic companion. The estimated reward-to-go for selecting action a in s_k is given by the expected value of a-priori-route policies obtained in all possible state s_{k+1} :

$$E[V^{\pi(\nu, s_{k+1})}(s_{k+1})|s_k, a] = \sum_{s_{k+1} \in S(s_k, a)} P\{s_{k+1}|s_k, a\} \times V^{\pi(\nu, s_{k+1})}(s_{k+1}), \quad (1)$$

where $V^{\pi(\nu, s_{k+1})}(s_{k+1})$ is the expected value of an a-priori-route policy $\pi(\nu)$ originating from state s_{k+1} . When the process occupies state s_k at decision epoch k , it selects an action $a \in A(s_k)$ such that the value of $R_k(s_k, a) + E[V^{\pi(\nu, s_{k+1})}(s_{k+1})|s_k, a]$ is maximized. Figure 2a provides a visual representation of the one-step rollout procedure.

For a state s_k and each feasible action $a \in A(s_k)$, one-step rollout executes the search heuristic $|S(s_k, a)|$ times to find an a-priori-route policy, which results in applying the heuristic a total of $\sum_{a \in A(s_k)} |S(s_k, a)|$ times to select an action at s_k . For a 10-customer instance, suppose that the traveler needs to decide between staying at the current location and going to one of the 9 other locations. If a decision epoch occurs every minute ($\delta_i = 1$) while waiting at the current location and there are four possible queue lengths for each of the going location, then the heuristic will be executed 39 times to select an action (3 of these correspond to the “stay” decision and $9 \times 4 = 36$

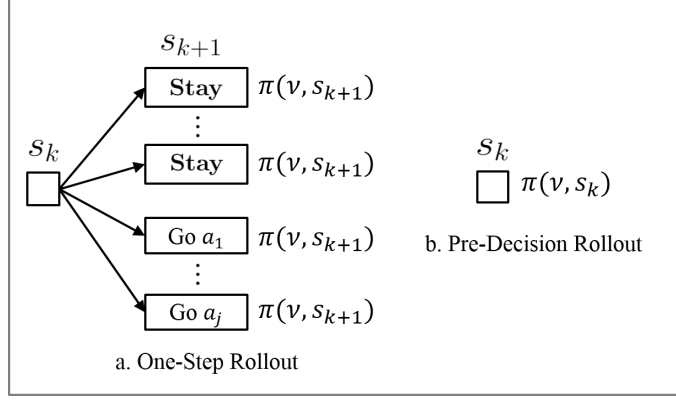


Figure 2: Rollout Decision Rule

correspond to the “go” decision). Notably, while staying at a customer, we need to execute heuristic search for 39 times at every minute unless some of the going actions are pruned according to Theorem 1. While the problem size and number of queue length realizations increase, $|A(s_k)|$ and $\sum_{a \in A(s_k)} |S(s_k, a)|$ increase, so selecting an action by evaluating $R_k(s_k, a) + E[V^{\pi(\nu, s_{k+1})}(s_{k+1}) | s_k, a]$ becomes computationally challenging even when determining the heuristic policy using local search and approximating $V^{\pi(\nu, s_{k+1})}(s_{k+1})$ using Monte Carlo sampling.

The formalization of the pre-decision rollout is motivated by the computational issues associated with one-step rollout (Goodson et al. (2016)). As shown in Figure 2b, the pre-decision state decision rule does not look ahead at all, but instead selects the action to take in state s_k via an a-priori-route policy $\pi(\nu, s_k)$ starting at the current location d_k and time t_k of state s_k . Specifically, the threshold value γ_d calculated for the current location d in the a priori route indicates how long to stay at this location. If $\gamma_d \neq 0$, the action selected in s_k is to stay at location d for a duration of γ_d . If $\gamma_d = 0$, then the action is to go to the first location specified in the a priori route $\pi(\nu, s_k)$ after leaving location d at time t . In this case, the VND heuristic is executed only once in each state s_k .

4.3 Compound Rollout

Because of the large state and action spaces induced by the waiting process, using one-step rollout, even when limiting per action evaluation run time to one minute and incorporating the action elimination presented in §3.3, to solve realistically sized instances is not computationally tractable. Thus, we propose a compound rollout algorithm to reduce the computational burden of one-step rollout and improve the policy quality of pre-decision rollout. As discussed in §3.1, if the traveler

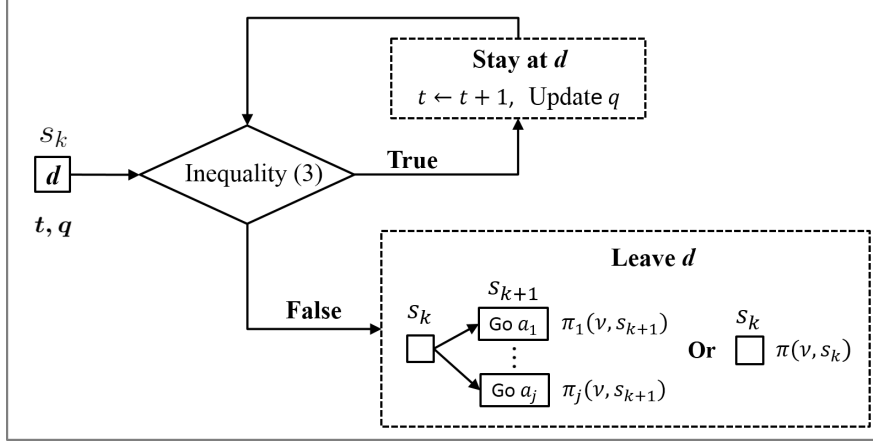


Figure 3: Compound Rollout

has not collected reward after arrival, she can either stay at the location or go to another location from which reward has not been collected. Compound rollout considers the stay-or-go decision in two stages. In the first stage, the traveler decides whether or not to stay at the current location. If the decision made in the first stage is to no longer stay at the current location, compound rollout then enters the second stage to determine which location to visit next. That is, we partition the action space $A(s_k)$ into two sets. One set is the singleton $\{d\}$ (if $d \in A(s_k)$), corresponding to staying and waiting at the current location. The other set is $\{A(s_k) \setminus \{d\}\}$, composed of actions of leaving and going to another location.

In the first stage of compound rollout, the traveler will stay at the current location if the expected total reward collected by staying at time t is greater than the expected total reward collected if the traveler departs at time t . In other words, the traveler will stay at the current location at time t if $V_{stay}(t) > V_{go}(t)$, where $V_{stay}(t)$ represents the expected reward-to-go if staying at the current location at time t and $V_{go}(t)$ denotes the expected reward-to-go if departing location d at time t and going to another location. We note that to compute $V_{stay}(t)$, we need to know the optimal decision of whether or not the traveler should leave at each of the following decision epochs. While the time of each future decision epoch is random, it is challenging to evaluate $V_{stay}(t)$ exactly. Thus, we approximate $V_{stay}(t)$ by assuming that if deciding to stay, the traveler will stay at the current location from time t till meeting with the customer or the end of the customer's time window. Specifically, the traveler will stay at the current location at time t if

$$P(\Omega_{dt} < l_d - t | q, t) \times r_d + \sum_{\omega_{dt} \leq l_d - t} P(\Omega_{dt} = \omega_{dt}) V_{go}(t + \omega_{dt} + m_d) > V_{go}(t), \quad (2)$$

where Ω_{dt} is the random variable representing the wait time at location d given the queue length q observed at time t . We derive the distribution of Ω_{dt} in §B of the electronic companion. We note that by assuming that the traveler will stay at a customer till meeting the customer or till the end of the customer's time window, we under-estimate the reward-to-go of the staying action. Because while deciding to stay at the current time t , the traveler may still be able to leave and go to another customer at each of the following decision epochs. However, as discussed earlier, it is not computational practical to evaluate the expected total reward of staying exactly. Thus, Inequality 2 is a heuristic for making the staying decision.

The comparison in Inequality (2) requires computing the estimate of $V_{go}(\cdot)$ over the entire support of Ω_{dt} . That is, each time when evaluating Inequality (2), we need to execute the search heuristic to find an a-priori-route policy for the number of times equal to the support of the wait time distribution. When deciding whether to stay, we need to evaluate the Inequality for every time increment until the Inequality does not hold. Therefore, it is too computationally expensive to use Inequality (2) directly in making the staying decision, and we replace Inequality (2) with

$$P(\Omega_{dt} < l_d - t | q, t) \times r_d + V_{go}(t + E[\Omega_{dt} | q, t]) > V_{go}(t). \quad (3)$$

We denote by $E[\Omega_{dt} | q, t]$ the expected wait time for observing queue length q at time t . We use $E[\Omega_{dt} | q, t]$ to estimate the actual wait time for the traveler. We note that at time $t + E[\Omega_{dt} | q, t]$, service may not actually occur. Thus, we do not include m_d in the computation of $V_{go}(t + E[\Omega_{dt} | q, t])$. To approximate the value of $V_{go}(t)$ and $V_{go}(t + E[\Omega_{dt} | q, t])$, we execute the VND heuristic onward from state s_k to find a-priori-route policies $\pi(\nu, s_k)$ and $\pi(\bar{\nu}, s_k)$ with expected value $V^{\pi(\nu, s_k)}(s_{k+1})$ and $V^{\pi(\bar{\nu}, s_k)}(s_{k+1})$, respectively. Notably, though both policies start at location d , the time to leave location d in policy $\pi(\nu, s_k)$ is t , and in policy $\pi(\bar{\nu}, s_k)$, it is $t + E[\Omega_{dt} | q, t]$.

As Figure 3 shows, the traveler will stay and wait at location d while Inequality (3) is true. If deciding to stay, she will wait for one time unit and re-evaluate the criteria. To do so, we advance the current time t by one, generate a realization of the queue length at time $t + 1$ (from the queue

length distribution derived in §A of the electronic companion), and re-evaluate Inequality (3) with the updated information. The traveler will leave location d once Inequality (3) does not hold. Then compound rollout enters the second stage to decide which location to go to next. Note that, if $q = 0$ at time t , the traveler collects reward and then leaves. In this case, the next location to visit is determined similarly to the second stage of our compound rollout.

As the lower-right hand corner of Figure 3 shows, in the second stage, compound rollout employs one-step rollout or pre-decision rollout to approximate the reward-to-go associated with candidates in the set $\{A(s_k) \setminus \{d\}\}$. In Algorithm 1, we formalize the action-selection logic used by the compound rollout algorithm. Line 1 and Line 2 indicate the criteria to stay at the current location d or to go to another location. If Inequality (3) indicates “go” rather than “stay”, compound rollout implements one-step rollout (Line 5) or pre-decision rollout (Line 6) to select the next location to visit after leaving location d . We note that while applying pre-decision rollout from state s_k to make “go” decision, the traveler will visit the first location specified in the a priori route $\pi(\nu, s_k)$ after d , i.e., ν_2 from $\nu = (\nu_1, \nu_2, \dots, \nu_m)$ with $\nu_1 = d$.

Algorithm 1: Compound Rollout

- 1: **if** $d \in A(s_k)$ and Inequality (3) is true **then**
 - 2: $a^* \leftarrow d$
 - 3: **else**
 - 4: Implement (i) one-step rollout or (ii) pre-decision rollout:
 - 5: (i) $a^* \leftarrow \arg \max_{a \in \{A(s_k) \setminus \{d\}\}} \{R_k(s_k, a) + E[V^{\pi(\nu, s_{k+1})}(s_{k+1})|s_k, a]\}$
 - 6: (ii) $a^* \leftarrow \nu_2$, where $\nu_2 \in \pi(\nu, s_k)$
-

5 Computational Experiments

We thank the reviewer for pointing out this issue. As discussed in Goodson et al. (2016), the performance of rollout algorithms can be bounded only when the heuristic policy is sequentially consistent heuristic from epoch to epoch. While we do not know of any sequentially consistent heuristic for the problem in this paper, Goodson et al. (2016) also shows to generate one using a concept known as fortification. However, because we are selecting between waiting decisions and routing decisions in this paper, we cannot fortify the heuristic used in this paper, whether in the one-step rollout or compound rollout. Thus, we are left to test our approach computationally.

In this section, we present computational results from applying the compound rollout procedure of §4.3. In §5.1, we detail the generation of problem instances for the DOPTW from existing benchmark problems. In §5.2, we provide implementation details, and in §5.3 and §5.4, we present and discuss our computational results.

5.1 Problem Instances

We derive our datasets from Solomon’s VRPTW instances (Solomon, 1987). We modify benchmark instances corresponding to randomly-located locations (R sets), clustered locations (C sets), and a mix of random and clustered locations (RC sets). In the experiments, we consider that the traveler typically visits between 10 and 20 locations and assume that the traveler will schedule 20 locations on each day. Accordingly, we select the first 20 locations from Solomon’s instances. For each instance, we maintain each location’s geographical and demand information as in Solomon (1987). We consider an eight-hour working day and assume the length of each location’s time window to be either 60 minutes, 90 minutes or 120 minutes. To create a 480-minute working day, we first scale time from the original time horizon in Solomon’s instances to a 480-minute day, $[0, 480]$. If the resulting time window is either 60 minutes, 90 minutes, or 120 minutes wide, then it is complete. If the resulting time window for location i is less than 60 minutes, we modify it to 60 minutes by setting $l_i = e_i + 60$. If the resulting time window for location i is between 60 minutes and 90 minutes wide, we modify it to 90 minutes by setting $l_i = e_i + 90$. If the resulting time window width for location i exceeds 90 minutes, we set it to 120 minutes by setting $l_i = e_i + 120$. These DOPTW data sets are available at http://ir.uiowa.edu/tippie_pubs/63.

5.2 Implementation and Details

We code our algorithms in C++ and execute the experiments on 2.6-GHz Intel Xeon processors with 64-512 GB of RAM. As mentioned in §4.1, we use a VND procedure to find the a-priori-route policies whose expected rewards are used to approximate the reward-to-go. In the VND, we estimate the expected value of heuristic policies using Monte Carlo sampling with 1000 samples. In this study, we consider one minute as the minimum time increment in the discrete-time Markov process. Thus, we use the best a-priori-route policy returned by the VND within a minute to approximate the reward-to-go. We determine the first location to visit by employing the VNS

from Zhang et al. (2014) as this first decision does not need to be solved within one minute. We implement the action elimination procedures presented in §3.3. For each problem instance, we randomly generate 1000 sample paths based on the transient queue length distribution presented in §A of the electronic companion.

Because exponential distributions with memoryless property are widely used to model the inter-arrival times and service times in queueing theory and geometric distributions are discrete analogue of exponential distributions, we assume X_i and Y_i , the competitors' interarrival and service times, respectively, are geometric random variables with parameters p_{x_i} and p_{y_i} , respectively. Further, we assume the distributions of X_i and Y_i are independent. We assume that the traveler and/or her associates have been able to collect information on her competitors' arrival and departure at locations, in which case the parameter values p_{x_i} and p_{y_i} are known to the traveler. For simplicity, we assume homogeneous inter-arrival and service time distributions in the study. For the geometric distributions of competitor arrivals and departures, we respectively set $p_{x_i} = 0.125$ and $p_{y_i} = 0.1$ for all locations $i \in V$. We derive the distributions of wait times in §B. We assume that the service time at each location i is deterministic as 10 minutes, i.e., $m_i = 10$ for all $i \in V$.

In the experiments, we consider the application of the problem to routing textbook sales representatives to visit professors on campus. Thus, a priority queue is assumed, in which the traveler has the lowest priority. We note, however, that a different queueing discipline only affects the wait time distributions that serve as inputs to our model and have no impact on our solution methodology.

5.3 Computational Results

To benchmark the performance of our rollout policies, we generate lower and upper bounds on the reward collected for each instance. To obtain a lower bound, we execute the a priori routes produced by the approach of Zhang et al. (2014). We establish an upper bound on each instance by solving it with the best-case assumption that the traveler experiences zero wait time at each location. As we already assume deterministic travel times between locations, the problem then becomes a deterministic orienteering problem. We employ the dynamic programming solution approach modified by Zhang et al. (2014) from Feillet et al. (2004) to solve this deterministic problem as an elementary longest path problem with resource constraints.

In our computational experiments, we solve the DOPTW with compound rollout using one-

Dataset	A Priori	Dynamic	CI	Gap	UB
R101	114.85	115.27	$[114.32, 116.23]$	0.37%	175
R102	131.11	133.65	$[132.90, 134.39]$	1.94%	203
R103	112.81	116.01	$[114.69, 117.34]$	2.84%	195
R104	104.71	110.10	$[108.71, 111.48]$	5.14%	189
R105	113.60	116.96	$[115.80, 118.13]$	2.96%	200
R106	125.82	130.59	$[129.70, 131.48]$	3.79%	221
R107	110.35	115.89	$[114.89, 116.88]$	5.02%	197
R108	101.65	106.76	$[105.69, 107.83]$	5.03%	189
R109	122.68	125.57	$[124.73, 126.42]$	2.36%	219
R110	110.06	117.65	$[116.75, 118.55]$	6.90%	189
R111	122.62	128.92	$[127.99, 129.85]$	5.14%	208
R112	92.23	95.60	$[94.56, 96.63]$	3.65%	177
R201	126.40	128.46	$[127.63, 129.29]$	1.63%	203
R202	124.46	132.32	$[131.37, 133.26]$	6.31%	215
R203	110.56	117.72	$[116.58, 118.85]$	6.47%	196
R204	98.95	107.66	$[106.44, 108.88]$	8.80%	189
R205	130.70	132.29	$[131.36, 133.22]$	1.22%	229
R206	126.64	135.16	$[134.10, 136.22]$	6.73%	230
R207	110.99	116.67	$[115.67, 117.66]$	5.11%	199
R208	99.57	106.50	$[105.38, 107.61]$	6.95%	189
R209	118.06	122.02	$[121.06, 122.97]$	3.35%	202
R210	127.64	132.19	$[131.20, 133.18]$	3.56%	228
R211	106.77	111.87	$[110.78, 112.95]$	4.77%	204

Table 1: DOPTW Results for R Instances with 20 Locations

step rollout in the second stage. Tables 1 through 3 compare the compound rollout policies to the bounds. In each of the three tables, the second and sixth columns report the lower and upper bounds obtained in the manner discussed in the previous paragraph. The third column reports the average objectives over 1000 sample paths from the compound rollout policies obtained by using one-step rollout to make the “go” decision. The fourth column presents the 95% confidence intervals on the dynamic objective values. The fifth column shows the gap between the a priori and the dynamic solutions ($Gap = \frac{Dynamic - A\ Priori}{A\ Priori} \times 100\%$).

Overall, the average objectives of compound rollout policies are 3.47% better than the a priori solutions. The average gap between the dynamic and a priori solutions for C instances is 2.56%, 4.35% for the R instances, and 3.15% for the RC instances. As shown in the fourth columns of the tables, for 54 out of 56 instances, the compound rollout policies are statistically better than the a priori solutions based on a 95% confidence level. For the remaining 2 instances (R101 and RC207),

Dataset	A Priori	Dynamic	CI	Gap	UB
C101	249.56	253.20	[252.22, 254.18]	1.46%	340
C102	212.69	216.94	[215.64, 218.24]	2.00%	330
C103	198.02	204.81	[203.46, 206.16]	3.43%	310
C104	168.00	176.84	[175.32, 178.36]	5.26%	280
C105	251.06	255.20	[254.30, 256.10]	1.65%	340
C106	248.74	255.11	[254.10, 256.12]	2.56%	340
C107	237.06	241.63	[240.70, 242.56]	1.93%	350
C108	235.37	238.86	[237.57, 240.15]	1.48%	360
C109	219.41	224.89	[223.71, 226.07]	2.50%	360
C201	288.25	291.08	[289.85, 292.31]	0.98%	360
C202	262.76	264.69	[263.76, 265.62]	0.74%	330
C203	208.85	212.62	[211.38, 213.86]	1.81%	300
C204	192.48	201.27	[199.91, 202.63]	4.57%	280
C205	290.80	295.58	[294.30, 296.86]	1.64%	350
C206	272.27	278.48	[277.02, 279.94]	2.28%	360
C207	233.21	247.59	[246.01, 249.17]	6.17%	360
C208	271.12	279.52	[278.52, 280.52]	3.10%	360

Table 2: DOPTW Results for C Instances with 20 Locations

Dataset	A Priori	Dynamic	CI	Gap	UB
RC101	170.46	177.14	[175.56, 178.72]	3.92%	260
RC102	207.8	214.13	[212.25, 216.01]	3.05%	330
RC103	191.07	199.24	[197.42, 201.06]	4.28%	320
RC104	174.4	183.64	[182.07, 185.21]	5.30%	300
RC105	203.71	207.42	[206.20, 208.64]	1.82%	300
RC106	180.07	188.60	[186.74, 190.46]	4.74%	320
RC107	165.37	173.61	[171.89, 175.33]	4.98%	290
RC108	175.6	177.05	[176.13, 177.97]	0.83%	300
RC201	215.48	219.63	[218.37, 220.89]	1.93%	300
RC202	218.13	223.24	[221.71, 224.77]	2.34%	340
RC203	205.2	207.44	[205.73, 209.15]	1.09%	330
RC204	168.84	176.52	[174.97, 178.07]	4.55%	300
RC205	210.95	220.65	[219.05, 222.25]	4.60%	350
RC206	213.18	219.65	[217.79, 221.51]	3.03%	370
RC207	196.41	195.49	[193.92, 197.06]	-0.47%	350
RC208	189.07	197.56	[195.86, 199.26]	4.49%	330

Table 3: DOPTW Results for RC Instances with 20 Locations

the compound rollout policies are not significantly different from the a priori solutions. In part, the lack of difference is due to the constraints imposed by these instances. In instance R101, every location has 60-minute time window, and because of this limited availability, no revisits occur in the compound rollout policies, which reduces the possible advantage of the dynamic solution over an a priori solution.

As mentioned in §5.2, we employ VNS to search for the a priori solutions. On average, the VNS requires around 23 minutes to converge. Given that we want to solve problems in real time, in the dynamic compound rollout approach, we only allow a VND heuristic one minute of search time per epoch according to the minimum time increment considered in our model. Running the VND within this restricted time may lead to inferior a-priori-route policies and thereby imprecise estimates of reward-to-go. In general, the rollout procedure helps overcome the minimal runtime afforded to the search heuristic as noted in Chang et al. (2013, p. 197) “... what we are really interested in is the ranking of actions, not the degree of approximation. Therefore, as long as the rollout policy preserves the true ranking of actions well, the resulting policy will perform fairly well.” However, for some instances, the values of the a priori route policies affect the choice of the next location to visit, thereby the quality of dynamic policies. For C instances, with clustered locations and overlapping time windows, the values of the a priori route policies used by the rollout approach to select the next location to visit are close together, and it is difficult to distinguish one from the other within the one-minute search time. We test instances C102 and C104 by approximating the reward-to-go with a-priori-route policies obtained via executing VND for up to five minutes. The average objective of C102 improves from 216.94 to 219.4 or by 1.13% and that of C104 improves from 176.84 to 178.57 or by 0.98%. In the case of instance RC207, in which there is a mixture of random and clustered locations, substantial runtime was required to distinguish one choice from another due to significant overlap in the time windows within a cluster. We test instance RC207 by obtaining a-priori-route policies via running VNS for up to 20 minutes and the average objective improves from 195.49 to 200.62, with a confidence interval of [198.96, 202.29], in which case the dynamic solution is statistically better than the a priori solution.

The upper bound obtained by solving the DOPTW with the assumption of zero wait time at locations is weak in all problem instances. The average gap between dynamic solutions and upper bounds is 56.58%, suggesting that the queueing effects at the locations is the complicating feature

Dataset	C.I. on Improv.	Avg.	Dataset	C.I. on Improv.	Avg.	Dataset	C.I. on Improv.	Avg.
R101	[2.46%, 4.77%]	3.62%	R109	[0.13%, 2.30%]	1.22%	R205	[3.06%, 5.21%]	4.14%
R102	[2.44%, 4.08%]	3.26%	R110	[2.69%, 4.94%]	3.82%	R206	[4.25%, 6.37%]	5.31%
R103	[-1.95%, 1.28%]	-0.34%	R111	[2.40%, 4.60%]	3.50%	R207	[2.60%, 4.94%]	3.77%
R104	[-1.20%, 2.41%]	0.61%	R112	[0.31%, 3.50%]	1.91%	R208	[4.80%, 7.78%]	6.29%
R105	[2.63%, 5.24%]	3.94%	R201	[0.48%, 2.37%]	1.43%	R209	[2.70%, 4.92%]	3.81%
R106	[3.07%, 5.04%]	4.06%	R202	[0.87%, 2.87%]	1.87%	R210	[1.58%, 3.68%]	2.63%
R107	[0.36%, 2.83%]	1.60%	R203	[0.07%, 2.74%]	1.41%	R211	[4.05%, 6.65%]	5.35%
R108	[-0.56%, 2.30%]	0.87%	R204	[4.01%, 7.30%]	5.66%			
C101	[-0.54%, 0.57%]	0.02%	C107	[0.39%, 1.52%]	0.96%	C204	[1.07%, 3.10%]	2.09%
C102	[-1.06%, 0.63%]	-0.22%	C108	[-1.04%, 0.51%]	-0.27%	C205	[0.12%, 1.39%]	0.76%
C103	[-1.24%, 0.65%]	-0.30%	C109	[-0.56%, 0.88%]	0.16%	C206	[2.70%, 4.13%]	3.42%
C104	[-1.65%, 0.86%]	-0.40%	C201	[0.50%, 1.63%]	1.07%	C207	[-0.34%, 1.48%]	0.57%
C105	[-0.52%, 0.54%]	0.01%	C202	[-0.41%, 0.60%]	0.10%	C208	[-0.35%, 0.69%]	0.17%
C106	[-0.25%, 0.87%]	0.31%	C203	[-0.22%, 1.44%]	0.61%			
RC101	[-0.28%, 2.24%]	0.98%	RC107	[5.02%, 8.02%]	6.52%	RC205	[0.01%, 2.06%]	1.04%
RC102	[3.81%, 6.35%]	5.08%	RC108	[2.04%, 3.64%]	2.84%	RC206	[-0.75%, 1.74%]	0.50%
RC103	[4.22%, 6.92%]	5.57%	RC201	[-0.14%, 1.51%]	0.69%	RC207	[1.49%, 3.86%]	2.68%
RC104	[-1.27%, 1.21%]	-0.03%	RC202	[0.28%, 2.35%]	1.32%	RC208	[-1.08%, 1.36%]	0.14%
RC105	[0.59%, 2.19%]	1.39%	RC203	[-1.35%, 0.88%]	-0.24%			
RC106	[-0.12%, 2.64%]	1.26%	RC204	[1.29%, 3.81%]	2.55%			

Table 4: Comparison between Compound-One-Step and Compound-Pre-Decision Rollout

in this problem. However, even for small size problems, it is computationally intractable to solve the DOPTW optimally to obtain an accurate upper bound.

To demonstrate the value of looking ahead and observing the queue length information at the next decision epoch while making the “go” decision, we compare our compound rollout policies using one-step rollout in making the “go” decision with those using pre-decision rollout. Specifically, line 5 in Algorithm 1 is implemented when using one-step rollout in the second stage, and line 6 is implemented when using pre-decision rollout. In the following discussion, we name the compound rollout procedure using one-step rollout in making “go” decision the *compound-one-step rollout* and that using pre-decision rollout the *compound-pre-decision rollout*. In Table 4, we provide detailed comparisons between the two types of policies. The second, fifth, and eighth columns present the 95% confidence intervals on the percentage of improvement that compound-one-step rollout policies have over compound-pre-decision rollout policies. The third, sixth, and ninth columns present the average improvements. While, on average, the compound-pre-decision rollout policies are 1.48% better than the a priori solutions, they are 1.98% worse than compound-one-step rollout policies. As shown in the table, out of 56 instances, there are 34 instances for which the compound-one-

step rollout policies are statistically better than the compound-pre-decision rollout policies. Out of these 34 instances, compound-one-step rollout has more than 2% of average improvement over compound-pre-decision rollout.

Though it is not computationally practical to solve instances of 20 customers using the one-step rollout algorithm, we compare the quality of one-step rollout and compound rollout solutions on 10-customer instances. In the compound rollout, we use one-step rollout to make going decisions. We present the results in Tables 5 through 7. In each of the three tables, the second column presents the objectives of a-priori-route policies. The third and sixth columns present the average objectives over 1000 sample paths from the one-step and compound rollout solutions, respectively. The fourth and seventh columns present the 95% confidence intervals on the objective values from the two types of policies, respectively. The eighth column presents the gaps between the a-priori-route and one-step rollout policies ($Gap_1 = \frac{OneStep - A\ Priori}{A\ Priori} \times 100\%$). The ninth column presents the gaps between the one-step and compound rollout policies ($Gap_2 = \frac{OneStep - Compound}{Compound} \times 100\%$).

On average, the one-step rollout policies are 1.01% better than the compound rollout policies and 3.57% better than the a priori policies. For 34 out of the 56 instances, the compound rollout policies are not significantly different from the one-step rollout policies. The average gaps between the one-step rollout policies and compound rollout policies for R, C, and RC instances are 0.33%, 1.38%, and 1.6%, respectively. We note that the difference between the one-step rollout and compound rollout lies in making staying decisions. As discussed in §4.3, while both Inequalities 2 and 3 underestimate the expected reward of staying at the current location, the traveler may leave earlier by following compound rollout policies than the one-step rollout policies. This may lead to inferior solution qualities of the compound rollout solutions compared with the one-step rollout solutions. When customers are randomly located as in the R instances, more staying decisions are made than when customers are clustered as in C and RC instances. Thus, the gap between one-step and compound rollout policies for R instances is small than those for the C and RC instances. The average gaps between one-step rollout policies and a-priori-route policies for R, C, and RC instances are 2.15%, 3.2%, and 6%, respectively. We note that although for 54 out of 56 instances, the one-step rollout policies are better than the a-priori-route policies, the a-priori-route policies are statistically better than the one-step rollout policies for instances C101 and C106. For both instances, the a-priori-route policy is 0, 5, 3, 7, 8, 10, 9, 6, 4, 2, and 1, following which the traveler

Dataset	A Priori	OneStep	CI	Avg. Pruned	Compound	CI	Gap ₁	Gap ₂
R101	87.41	87.47	[87.17, 87.77]	0	87.44	[87.14, 87.74]	0.06%	0.03%
R102	82.14	84.25	[83.59, 84.91]	81	83.68	[82.98, 84.37]	2.56%	0.68%
R103	82.14	84.43	[83.77, 85.1]	82	83.73	[83.01, 84.45]	2.78%	0.84%
R104	72.60	75.27	[74.63, 75.9]	53	75.3	[74.57, 76.02]	3.68%	-0.04%
R105	88.08	88.03	[87.74, 88.33]	0	87.9	[87.6, 88.19]	-0.05%	0.25%
R106	83.17	85.18	[84.57, 85.79]	83	85.06	[84.39, 85.72]	2.42%	0.14%
R107	83.17	85.33	[84.73, 85.93]	82	84.78	[84.11, 85.45]	2.60%	0.65%
R108	72.92	75.67	[75.01, 76.34]	52	75.95	[75.2, 76.7]	3.77%	-0.37%
R109	81.37	84.42	[84.07, 84.78]	113	83.2	[82.85, 83.54]	3.75%	1.47%
R110	80.65	82.73	[82.04, 83.43]	8	81.78	[81.05, 82.51]	2.58%	1.16%
R111	91.64	92.19	[91.89, 92.5]	33	92.51	[92.23, 92.8]	0.60%	-0.35%
R112	74.81	74.95	[74.32, 75.58]	0	75.5	[74.84, 76.16]	0.19%	-0.73%
R201	87.02	87.06	[86.75, 87.36]	0	87.03	[86.72, 87.33]	0.04%	0.16%
R202	80.88	82.86	[82.15, 83.58]	82	82.97	[82.21, 83.73]	2.44%	-0.13%
R203	80.88	82.8	[82.12, 83.49]	84	82.95	[82.2, 83.71]	2.37%	-0.18%
R204	73.84	76.61	[75.92, 77.3]	54	75.88	[75.14, 76.63]	3.75%	0.96%
R205	91.98	92.15	[91.84, 92.46]	14	92.31	[91.92, 92.71]	0.18%	0.22%
R206	85.13	85.73	[85.12, 86.33]	90	86.39	[85.83, 86.95]	0.70%	-0.76%
R207	85.13	85.76	[85.16, 86.37]	91	86.46	[85.91, 87.01]	0.74%	-0.81%
R208	75.51	76.84	[76.07, 77.61]	69	76.9	[76.15, 77.65]	1.77%	-0.08%
R209	84.32	90.46	[89.79, 91.13]	91	87.55	[86.85, 88.25]	7.28%	3.32%
R210	102.84	103.99	[103.81, 104.18]	40	102.91	[102.71, 103.12]	1.12%	1.05%
R211	81.66	85.04	[84.46, 85.63]	57	84.38	[83.78, 84.98]	4.14%	0.78%

Table 5: Results for R instances with 10 customers

can collect reward at each customer. However, after visiting customer 6 in the dynamic policies, the traveler may either go to customer 4 or go to customer 2, depending on the reward-to-go approximations. If going to customer 2 after customer 6, the traveler may not be able to collect as much reward as going to customer 4 first after customer 6. We note that while the reward-to-go is approximated by a-priori-route policies in the dynamic policy, the decisions made may be affected by the quality of the approximation.

The fourth column of the table presents the average number of actions eliminated in the one-step rollout policies, by implementing Theorem 1 and Theorem 2. We note that when making staying decision via one-step rollout, we need to evaluate the action of staying together with all the going actions. For each of the going actions and each queue length realization, we need to execute the search heuristic to find an a-priori-routing policy. By eliminating some of the going actions, we save the computational efforts in making staying decision while developing one-step rollout routing policies.

Dataset	A Priori	OneStep	CI	Avg. Pruned	Compound	CI	Gap ₁	Gap ₂
C101	150.00	148.11	[147.87, 148.35]	1	147.97	[147.72, 148.22]	-1.26%	0.09%
C102	119.21	128.92	[128.16, 129.68]	118	124.71	[123.81, 125.61]	8.15%	3.38%
C103	119.21	129.41	[128.66, 130.16]	114	123.63	[122.78, 124.48]	8.56%	4.68%
C104	96.45	108.86	[107.91, 109.81]	46	106.87	[105.81, 107.93]	12.87%	1.86%
C105	145.00	145.85	[145.49, 146.21]	84	144.71	[144.32, 145.1]	0.59%	0.79%
C106	150.00	147.74	[147.48, 148.0]	0	147.81	[147.55, 148.07]	-1.51%	-0.05%
C107	144.16	145.1	[144.72, 145.48]	62	143.97	[143.57, 144.37]	0.65%	0.88%
C108	144.16	145.09	[144.72, 145.46]	79	143.98	[143.58, 144.38]	0.65%	0.78%
C109	136.68	141.53	[141.06, 142.0]	123	139.94	[139.43, 140.45]	3.55%	1.14%
C201	129.51	131.13	[130.42, 131.84]	37	127.66	[126.92, 128.4]	1.25%	3.30%
C202	101.21	104	[103.52, 104.48]	36	101.88	[101.36, 102.4]	2.76%	2.08%
C203	101.21	103.44	[102.93, 103.95]	38	101.89	[101.38, 102.4]	2.21%	1.52%
C204	88.11	93.95	[93.18, 94.72]	42	93.93	[93.0, 94.86]	6.63%	0.02%
C205	129.42	132.07	[131.41, 132.73]	43	128.78	[128.06, 129.5]	2.05%	2.79%
C206	119.63	121.94	[121.22, 122.66]	107	119.84	[119.1, 120.58]	1.93%	1.75%
C207	123.34	124.41	[124.05, 124.77]	13	123.98	[123.6, 124.36]	0.87%	0.35%
C208	113.33	118.37	[117.77, 118.97]	93	118.72	[118.15, 119.29]	4.44%	-0.29%

Table 6: Results for C instances with 10 customers

Dataset	A Priori	OneStep	CI	Avg. Pruned	Compound	CI	Gap ₁	Gap ₂
RC101	158.11	159.35	[158.1, 160.6]	38	159.34	[158.09, 160.59]	0.79%	1.11%
RC102	135.85	147.35	[145.88, 148.82]	114	143.79	[142.21, 145.37]	8.46%	2.48%
RC103	135.85	147.25	[145.74, 148.76]	113	145.16	[143.61, 146.71]	8.39%	1.44%
RC104	122.28	138.05	[136.8, 139.3]	62	135.92	[134.6, 137.24]	12.89%	1.57%
RC105	173.22	179.64	[179.02, 180.26]	71	178.18	[177.57, 178.79]	3.70%	0.82%
RC106	149.37	162.25	[160.97, 163.53]	53	156.99	[155.56, 158.42]	8.62%	3.35%
RC107	140.97	152.66	[151.53, 153.79]	28	152.59	[151.3, 153.88]	8.29%	0.05%
RC108	173.58	176.3	[175.48, 177.12]	5	174.6	[173.73, 175.47]	1.57%	0.97%
RC201	183.58	184.11	[183.44, 184.78]	11	183	[182.34, 183.66]	0.29%	0.61%
RC202	149.67	163.6	[162.37, 164.83]	126	157.3	[156, 158.6]	9.31%	4.01%
RC203	149.67	163.23	[162.03, 164.43]	123	157.66	[156.36, 158.96]	9.06%	3.53%
RC204	122.26	136.16	[134.96, 137.36]	60	135.81	[134.46, 137.16]	11.37%	0.26%
RC205	164.63	172.16	[171.29, 173.03]	159	170.1	[169.07, 171.13]	4.58%	1.21%
RC206	182.22	187.46	[186.64, 88.28]	10	186.16	[185.32, 187]	2.88%	0.70%
RC207	163.55	166.53	[165.57, 167.49]	167	163.1	[162.05, 164.15]	1.82%	2.29%
RC208	167.47	174.2	[173.07, 175.33]	141	169.84	[168.66, 171.02]	4.02%	2.57%

Table 7: Results for RC instances with 10 customers

5.4 Policy Analysis

To illustrate the differences between a priori solutions and our compound rollout policies, we provide a detailed comparison of select instances. In the comparison, the compound rollout policies are obtained by using one-step rollout in making the “go” decision. Overall, the advantage of dynamic solutions over a priori solutions is that dynamic solutions select actions based on the realized random information, which may enable the traveler to adapt to the observed queues, e.g., visiting more or different locations by taking advantage of favorable realizations or revisiting a location. In contrast, a priori solutions specify a fixed sequence of locations that maximizes the collected reward averaged over all possible scenarios. Further, it is difficult to construct a fixed sequence to facilitate revisiting locations as this action comes as a reaction to a specific queue length observations.

We compare a priori solutions and compound rollout policies for instances R107 and C204 in Figures 4 and 5. In each figure, the location indices are ordered according to the a priori route. For each location, the left bar corresponds to the a priori solution and the right bar represents data from the compound rollout policy. However, we note that the sequence of locations in a dynamic solution does not necessarily correspond to the a priori sequence as the dynamic approach adjusts the sequence based on realized observations. The overall height of each bar shows the probability of visiting a location. For a priori solutions, we compute the probability of the traveler visiting but not collecting reward from a location and the probability of collecting reward via the analytical formulation of Zhang et al. (2014). For dynamic policies, the probabilities of visiting but not collecting reward, collecting reward on the first visit, and collecting reward by revisiting are computed via simulation with 1000 sample paths.

For instance R107 with 20 locations, the a priori route is (5, 19, 11, 9, 16, 14, 18, 20, 2, 15, 1, 7, 10, 4, 13, 17). Note that locations not visited by the traveler in the a priori or dynamic solutions are not listed here. From Figure 4, we can see that when locations are randomly located, the a priori solution is only able to visit 8 out of 20 locations, while dynamic solution is likely to visit 15 locations. Both the a priori and dynamic solutions are able to collect reward from locations 5, 11, 9, and 10 with certainty or near certainty. The a priori solution is more likely to visit and collect reward from locations 19 and 4, and has around 45% of likelihood of visiting and 15% of likelihood of collecting reward from location 17, who is skipped in the dynamic solution. However,

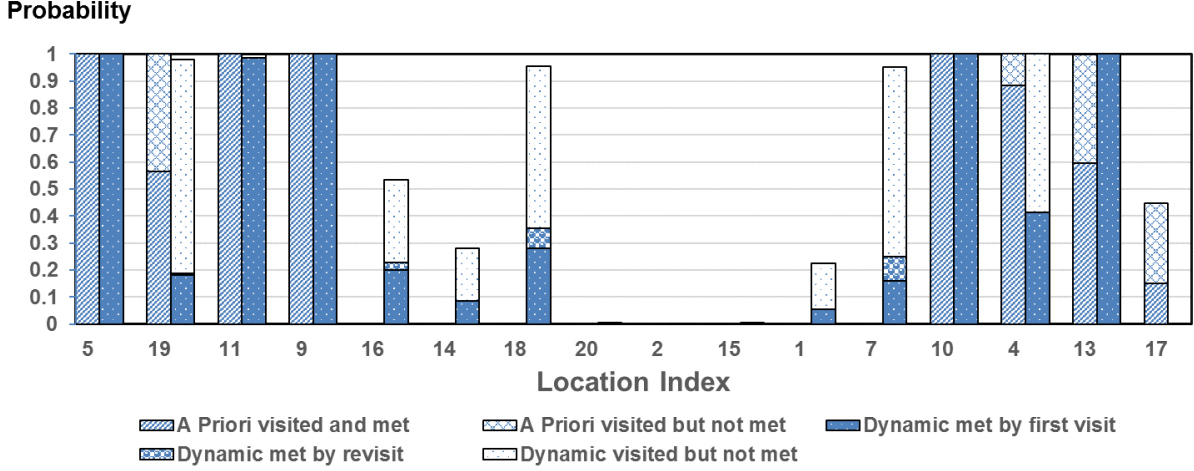


Figure 4: Compound Rollout vs. A Priori for Dataset R107

the dynamic solution is more likely to collect reward from location 13, who has the second largest reward of all locations, and has a positive probability of visiting locations 16, 14, 18, 20, 2, 15, 1, 7, who are always skipped by the a priori solution. By allowing revisiting, the dynamic solution increases the likelihood of collecting rewards at several locations. For instance, revisiting leads to around a 10% of increase in the likelihood of collecting reward from location 7, a more than 5% from location 18, and around a 3% of increase from location 16.

Similarly, for instance C204, the a priori route is (16, 14, 12, 19, 18, 17, 10, 5, 2, 1, 7, 3, 4, 15, 13, 9, 11, 8), where locations skipped in both the a priori and dynamic tours are not listed. We note that the traveler is able to visit and collect reward from more locations when locations are clustered than randomly located as in the R instances. As Figure 5 shows, the traveler always visits and collects rewards from locations 16, 4, 15, 13, 9, and 8 in both the a priori and dynamic solutions. Both a priori and dynamic solutions always visit location 12 and a priori solution is more likely to visit locations 10, 5, and 2, but both solutions have similar likelihoods of collecting rewards from locations 12, 10, 5, and 2. The a priori solution is more likely to visit and collect rewards from locations 1, 7, and 3. However, the overall performance of the dynamic solution is better because it has a higher probability of collecting rewards from locations 14, 19, 18, 17, and 11, which together provide more reward compared to locations 10, 5, and 2. Revisits enable the dynamic solution to increase the likelihood of collecting rewards from locations 14, 12, 19, 18, 17, and 11. Specifically, revisits to locations 14, 19, and 18 in the dynamic solution lead to over 10% of

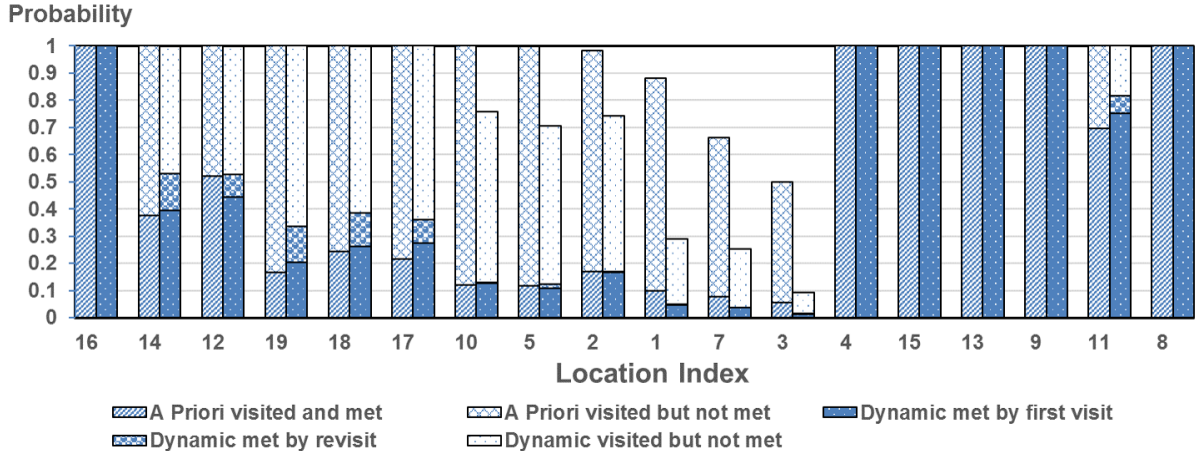


Figure 5: Compound Rollout vs. A Priori for Dataset C204

increase in the likelihood of collecting rewards from these locations, while increasing the likelihood of collecting rewards from locations 12, 17, and 11 by up to 8%. The likelihood of collecting reward from location 12 by the first visit in the dynamic solution is lower than that of the a priori solution. However, revisits make the overall probability of collecting reward from location 12 from the dynamic solution greater than that of the a priori solution.

6 Summary and Future Work

Motivated by the daily routing problem faced by a traveler, we introduce a dynamic orienteering problem with time windows (DOPTW) characterized by a queueing process and a time window at each location. Specifically, the queueing process induces random wait times at locations. To generate dynamic policies that govern routing and queueing decisions, we propose a novel compound rollout algorithm that explicitly partitions the action space and executes rollout policies based on the partitioning. Our dynamic policies perform favorably in comparison to the a priori routing solutions with recourse actions. We demonstrate the value of incorporating queueing information in making “where to go” decision by comparing the compound rollout using one-step rollout to make the “go” decision to that using pre-decision rollout which does not look ahead when selecting a location to go next.

On average, our compound-one-step rollout policies perform about 3.47% better than the a priori solutions, which suggests the merit in solving the DOPTW dynamically. By looking ahead

and involving future queue information in approximating reward-to-go when making “where to go” decision, our compound-one-step rollout policies are preferable to the compound-pre-decision rollout policies by having an average improvement of 1.98% on objective values. However, compound-pre-decision rollout is a viable alternative if computation becomes a concern.

We suggest several extensions of the current work. One is to consider that the traveler may need to compete with others for the rewards from customers. For example, textbook sales representatives compete for adoptions from professors. A textbook salesperson may visit a professor several times during a multi-period horizon, trying to develop a stronger relationship that leads to greater chance of adoption. We believe that this extension can be approached as a multi-period orienteering problem. The second extension is to investigate other approaches in making the staying decision. As discussed in §4.3, due to the increased computation required to exactly compute the reward-to-go of a staying action, our current approach underestimated the value of staying at a customer. A better way of evaluating the staying action would improve the overall performance of the compound rollout policies. Third, although no theoretical guarantees can be provided on the performance of the rollout policies, it would be interesting to develop a solution approach in which we can demonstrate performance guarantees for this problem.

A State Transition Probabilities

In this section, we derive the state transition probabilities, $P\{s_{k+1}|s_k, a\}$, for the various state-action pairs. For both the decision to stay at the current location and the decision to go to another location, the transition probabilities depend on the evolution of the queue at the respective location. As mentioned in §3.1, the evolution of the queue length is governed by competitor arrivals and competitor departures upon completing service at the location. We define X_i as the random variable representing the competitors’ inter-arrival time at location i and x_i is a realization of X_i . We define Y_i as the random variable representing the service time of a competitor at location i and y_i is a realization of Y_i . We consider a discrete-time Markov model and assume X_i and Y_i are geometric random variables with parameters p_{x_i} and p_{y_i} , respectively. We assume the distributions of X_i and Y_i are independent. Let Q_{it} be the random variable representing the queue length observed at location i at time t and q_{it} be the realization of Q_{it} . We model the evolution of queue

at location i as a discrete-time Markov Chain (DTMC) $\{Q_{it}, t = 0, 1, 2, \dots, T\}$. We assume that $\{Q_{it}, t = 0, 1, 2, \dots, T\}$ and $\{Q_{jt}, t = 0, 1, 2, \dots, T\}$ are independent Markov chains for $i \neq j$.

Given $0 < q_{it} < L$, the possible realizations of $q_{i,t+1}$ are:

- $q_{i,t+1} = q_{it} + 1$ with probability $p_{x_i}(1 - p_{y_i})$ if there is only a competitor arrival and no competitor departure occurring at time $t + 1$;
- $q_{i,t+1} = q_{it} - 1$ with probability $p_{y_i}(1 - p_{x_i})$ if there is only a competitor departure and no competitor arrival at time $t + 1$;
- $q_{i,t+1} = q_{it}$ with probability $(1 - p_{x_i})(1 - p_{y_i}) + p_{x_i}p_{y_i}$ if there are no queueing events or both a competitor arrival and a departure at time $t + 1$.

If $q_{it} = 0$, then $q_{i,t+1} = 1$ or $q_{i,t+1} = 0$ with probabilities p_{x_i} and $1 - p_{x_i}$, respectively. If $q_{it} = L$, then $q_{i,t+1} = L - 1$ with probability p_{y_i} and $q_{i,t+1} = L$ with probability $1 - p_{y_i}$. Consequently, the DTMC on state space $\{0, 1, 2, \dots, L\}$ is described by the following time-homogeneous one-step transition matrix:

$$R_i = \begin{bmatrix} 1 - p_{x_i} & p_{x_i} & 0 & \cdots & 0 & 0 & 0 \\ (1 - p_{x_i})p_{y_i} & \bar{p}^i & (1 - p_{y_i})p_{x_i} & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & (1 - p_{x_i})p_{y_i} & \bar{p}^i & (1 - p_{y_i})p_{x_i} \\ 0 & 0 & 0 & \cdots & 0 & p_{y_i} & 1 - p_{y_i} \end{bmatrix},$$

where $\bar{p}^i = (1 - p_{y_i})(1 - p_{x_i}) + p_{x_i}p_{y_i}$.

From a state $s_k = (t, d, q, H, U, W, (\check{t}, \check{q}))$, consider the transition to a state s_{k+1} when deciding to leave the current location d and go to another location z . As mentioned in §3.1, the only random element of this transition is the queue length observed at location z at time $t + c_{dz}$. Therefore, the stochastic transition from s_k to s_{k+1} is governed by the queue length distribution.

If $z \in U$, the distribution of queue lengths observed at time $t + c_{dz}$ is defined over $[0, L]$, by $\phi_0 R_z^{(\max\{t+c_{dz}, e_z\}-o_z)}$, where ϕ_k is a vector with a one in the position corresponding to queue length k (≥ 0), $R_z^{(n)}$ is the n -step transition matrix, and o_z ($\leq e_z$) is the earliest time that a queue (of competitors) begins to form at location z . If $z \in W$, the distribution of queue lengths observed at time $t + c_{dz}$ is defined over $[0, L]$, by $\phi_{\check{q}_z} R_z^{(\max\{t+c_{dz}, e_z\}-\max\{\check{t}_z, o_z\})}$. Note that $\check{t}_z$ is the time of the most recent visit to location z and $\check{q}_z$ is the queue length observed at time $\check{t}_z$.

Now consider the transition to a state s_{k+1} from a state $s_k = (t, d, q, H, U, W, (\check{t}, \check{q}))$ when deciding to stay at the current location d . As mentioned in §3.1, the next decision epoch that is triggered by a queueing event or δ_i , the maximum amount of time between epochs, occurs at time $t + \min\{X_d, Y_d, \delta_i\}$. Thus, both the queue length and the system time of the next epoch is random. The state transition probability $P\{s_{k+1}|s_k, d\}$ is defined over $3\delta_i$ possible states characterized by $q' \in \{q+1, q, q-1\}$ and $t' \in \{t+1, t+2, \dots, t+\delta_i\}$. Let κ be the realization of $\min\{X_d, Y_d, \delta_i\}$. When deciding to stay at the current location, the time of next decision epoch is $t + \kappa$ ($0 < \kappa \leq \delta_i$). For $\kappa < \delta_i$, the event triggering the epoch at $t + \kappa$ is:

- (i) a competitor arrival increasing queue length by one with probability $p_{x_d}(1-p_{x_d})^{\kappa-1}(1-p_{y_d})^\kappa$;
- (ii) a competitor departure decreasing queue length by one with probability $p_{y_d}(1-p_{y_d})^{\kappa-1}(1-p_{x_d})^\kappa$;
- (iii) the concurrent arrival and departure so that queue length remains the same as in state s_k^a with probability $p_{x_d}p_{y_d}[(1-p_{x_d})(1-p_{y_d})]^{\kappa-1}$;

For $\kappa = \delta_i$, events (i), (ii), and (iii) can occur with the associated probabilities. Additionally, with probability $(1-p_{x_d})^{\delta_i}(1-p_{y_d})^{\delta_i}$, no queueing events may occur before or at time $t + \delta_i$ so that the queue length remains the same as in state s_k .

B Distribution of Wait Time

In this section, we derive the distribution of wait time Ω_{it} at location i if observing queue length q_{it} at time t . In our derivation of the wait time distributions, we assume a priority queue discipline in which the traveler has the lowest priority. Assuming another discipline would affect that distributions, but would not alter our solution approach nor the model. As the traveler has the lowest priority, Ω_{it} is the time it takes the observed queue q_{it} to diminish to zero. Thus, the wait time for q_{it} at location i is the first passage time from state q_{it} to state 0 for the DTMC described by the one-step transition matrix R_i in §A. Let $f_{q0}^i(n)$ be the probability that the first passage from state q ($0 \leq q \leq L$) to state 0 occurs after n periods at location i . Let $P_{qj}^i(n)$ be the probability that starting from state q , the DTMC is in state j after n periods at location i . As presented in

Scholtes (2001), based on the total probability formula, we have

$$P_{q0}^i(n) = P_{00}^i(n-1)f_{q0}^i(1) + \dots + P_{00}^i(1)f_{q0}^i(n-1) + f_{q0}^i(n). \quad (4)$$

Then $f_{q0}^i(n)$ can be computed recursively via:

$$\begin{aligned} f_{q0}^i(1) &= P_{q0}^i(1) \\ &\vdots \\ f_{q0}^i(n) &= P_{q0}^i(n) - f_{q0}^i(1)P_{00}^i(n-1) - \dots - f_{q0}^i(n-1)P_{00}^i(1). \end{aligned} \quad (5)$$

For queue length q_{it} observed at time t at location i , the distribution of Ω_{it} is defined over $[\max\{e_i - t, 0\}, l_i - t]$, by the q_i th row of the following matrix

$$\begin{bmatrix} f_{00}^i(1) & f_{00}^i(2) & \dots & f_{00}^i(l_i - t - 1) & \bar{f}_{00}^i \\ f_{10}^i(1) & f_{10}^i(2) & \dots & f_{10}^i(l_i - t - 1) & \bar{f}_{10}^i \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ f_{L-1,0}^i(1) & f_{L-1,0}^i(2) & \dots & f_{L-1,0}^i(l_i - t - 1) & \bar{f}_{L-1,0}^i \\ f_{L0}^i(1) & f_{L0}^i(2) & \dots & f_{L0}^i(l_i - t - 1) & \bar{f}_{L0}^i \end{bmatrix},$$

where $\bar{f}_{q0}^i = 1 - \sum_{k=1}^{l_i-t-1} f_{q0}^i(k)$ for $q = 0, \dots, L$.

C Structural Results and Proofs

For many decision-making problems under uncertainty, there exist optimal control-limit or threshold-type policies. In this section, we explore the existence of threshold queueing policies. We demonstrate that, while visiting locations following a fixed sequence and allowing no revisits to previously visited locations, there exist a control limit in queue length at any given time for each location. That is, if it is optimal to stay at a location at time t when observing a queue length q , it is also optimal to stay when observing a shorter queue length q' ($q' < q$) at the same time t . The proposition is stated in Theorem 3. We note that Theorem 3 holds regardless of the queueing discipline, though the threshold value may depend on the discipline.

Theorem 3. *Let q_{it} be the queue observed by the traveler at location i at time t . For a fixed sequence of locations, there exists a threshold q_{it}^* such that if $q_{it} \geq q_{it}^*$, then it is optimal for the traveler to leave the queue at location i , and if $q_{it} < q_{it}^*$, then it is optimal to stay in queue at location i .*

We denote by $g_i(q'|q, t)$ the distribution of the current queue length at location i given the queue length q observed t time units ago. According to the transient queue length distribution presented in §A, the distribution $g_i(q'|q, t)$ is given by the q th row of the t -step transition matrix $R_i^{(t)}$. Let $Stay(i, t, q)$ be the expected reward of staying and waiting at location i with queue length q at time t . Let $Leave(i, t, q)$ be the onward expected reward of leaving and going to the next location $i + 1$ in the sequence when there is a queue length q at location i at time t . The formulations of $Stay(i, t, q)$ and $Leave(i, t, q)$ are:

$$\begin{aligned} Stay(i, t, q) &= \sum_{q'} g_i(q'|q, 1) v(i, t + 1, q'), \\ Leave(i, t, q) &= \sum_{q'} P(Q_{i+1, t+c_{i,i+1}} = q') v(i + 1, t + c_{i,i+1}, q'). \end{aligned} \quad (6)$$

Note that the queue length at location $i + 1$ at time $t + c_{i,i+1}$ is independent with that at location i at time t . Let $v(i, t, q)$ be the expected reward from location i and the onward reward from other locations on the route if the traveler observes queue q at location i at time t . By definition, $v(i, t, q)$ can be represented as:

$$v(i, t, q) = \max \{ Stay(i, t, q), Leave(i, t, q) \}. \quad (7)$$

Before proving Theorem 3, we state and prove a series of lemmas.

Lemma 1. *The distribution of the queue length has the stochastic ordering property, i.e., $\sum_{q=k}^L g_i(q|\tilde{q}, t) \leq \sum_{q=k}^L g_i(q|q', t)$ for all i, k, t , and $q' > \tilde{q}$.*

Proof. Proof of Lemma 1 We prove this lemma by induction. First, by the definition of R_i in §A, it is true that $\sum_{q=k}^L g_i(q|\tilde{q}, 1) \leq \sum_{q=k}^L g_i(q|q', 1)$ for all k and $q' > \tilde{q}$ ($\tilde{q}, q' \in [0, L]$). Assuming $\sum_{q=k}^L g_i(q|\tilde{q}, n) \leq \sum_{q=k}^L g_i(q|q', n)$ for all k and $q' > \tilde{q}$, now we show $\sum_{q=k}^L g_i(q|\tilde{q}, n + 1) \leq$

$\sum_{q=k}^L g_i(q|q', n+1)$ for all k and $q' > \tilde{q}$. Then,

$$\sum_{q=k}^L g_i(q|\tilde{q}, n+1) = \sum_{q=k}^L \sum_{r=0}^L g_i(q|r, n) g_i(r|\tilde{q}, 1) \quad (8)$$

$$\leq \sum_{q=k}^L \sum_{r=0}^L g_i(q|r, n) g_i(r|q', 1) \quad (9)$$

$$= \sum_{q=k}^L g_i(q|q', n+1), \quad (10)$$

Equalities (8) and (10) follow the definition of n -step transition probability. Inequality (9) results from the induction hypothesis. $\square$

Lemma 2. For each location i , $v(i, t, \tilde{q}) \geq v(i, t, q')$, for all i, t and $q' > \tilde{q}$.

Proof. Proof of Lemma 2 We prove the lemma by induction. First, we show that, with $T = \max_{i \in V} l_i$, $v(i, T, \tilde{q}) \geq v(i, T, q')$ for all i and $q' > \tilde{q}$. By definition, when there is a queue at time T , $Stay(i, T, q) = 0$ and $Leave(i, T, q) = 0$. Therefore, $Stay(i, T, q)$ and $Leave(i, T, q)$ are non-increasing in q . As the the maximum of non-increasing functions is non-increasing, $v(i, T, \tilde{q}) \geq v(i, T, q')$ for $\tilde{q} < q'$.

Assuming $v(i, t+1, \tilde{q}) \geq v(i, t+1, q')$ for all i and $q' > \tilde{q}$, we next show that $v(i, t, \tilde{q}) \geq v(i, t, q')$ for all i and $q' > \tilde{q}$. We have:

$$\begin{aligned} & Stay(i, t, \tilde{q}) - Stay(i, t, q') \\ &= \sum_q g_i(q|\tilde{q}, 1) v(i, t+1, q) - \sum_q g_i(q|q', 1) v(i, t+1, q) \end{aligned} \quad (11)$$

$$\geq 0. \quad (12)$$

Equality (11) follows by definition. By the induction hypothesis, $v(i, t+1, q)$ is non-increasing in q . By Lemma 1, $\sum_{q=k}^L g_i(q|q', 1)$ is a non-decreasing function of q' for all k . Thus, $\sum_{q=k}^L g_i(q|q', 1) v(i, t+1, q)$ is non-increasing in q' , thereby implying Inequality(12). By definition, $Leave(i, t, \tilde{q}) = Leave(i, t, q')$ because the queue length at location i has no bearing on the queue length at location $i+1$. So both $Stay(i, t, q)$ and $Leave(i, t, q)$ are non-increasing on q for all i and t . Thus $v(i, t, q)$ is non-increasing on q for all i and t , i.e., $v(i, t, \tilde{q}) \geq v(i, t, q')$ for all i, t and $q' > \tilde{q}$. $\square$

Proof. Proof of Theorem 3 We prove the result by showing that, if it is optimal to leave at time t while observing queue length q_{it}^* , it is also optimal to leave at time t while observing queue length $q_{it} > q_{it}^*$. Suppose it is optimal to leave at time t with queue length q_{it}^* , then

$$\begin{aligned} & \text{Leave}(i, t, q_{it}^*) - \text{Stay}(i, t, q_{it}^*) \\ &= \sum_q P(Q_{i+1, t+c_{i,i+1}} = q) v(i+1, t+c_{i,i+1}, q) - \sum_q g_i(q|q_{it}^*, 1) v(i, t+1, q) \\ &\geq 0. \end{aligned} \tag{13}$$

Now we show that it is also optimal to leave at t with $q_{it} > q_{it}^*$. We have:

$$\begin{aligned} & \text{Leave}(i, t, q_{it}) - \text{Stay}(i, t, q_{it}) \\ &= \sum_q P(Q_{i+1, t+c_{i,i+1}} = q) v(i+1, t+c_{i,i+1}, q) - \sum_q g_i(q|q_{it}, 1) v(i, t+1, q) \end{aligned} \tag{14}$$

$$\geq \sum_q P(Q_{i+1, t+c_{i,i+1}} = q) v(i+1, t+c_{i,i+1}, q) - \sum_q g_i(q|q_{it}^*, 1) v(i, t+1, q) \tag{15}$$

$$\geq 0. \tag{16}$$

Given Lemmas 2, this result is straightforward. Therefore, it is also optimal to leave location i at t when $q_{it} > q_{it}^*$. $\square$

We note that it is not obvious how to calculate the value of q_{it}^* from Theorem 3. Besides, even when the sequence of locations is fixed, there does not exist a control limit with respect to the traveler's arrival time at a location. Even if it is optimal to stay at a location i when the traveler arrives at time t and observes a queue length q , it may be optimal to leave if she arrives at an earlier time $t' < t$ and observes the same queue length. The lack of control limit structure with respect to arrival time is due to the fact that the traveler may be more likely to collect rewards from locations later in the visit sequence if she leaves the current location at an earlier time.

We next provide an example showing that, given a queue, there does not exist a control limit with respect to the traveler's arrival time at a location. We consider a case with two locations. The first location is available during time window $[0, 40]$ and associated with a reward of 30, while the second location is available during time window $[0, 60]$ and with a reward of 50. Assuming the

queue length observed at location 1 upon arrival is 2, we show that although it is optimal for the traveler to join the queue at location 1 when she arrives at time 25, it is not optimal to join when she arrives at an earlier time 10.

Table 8 and 9 detail the counter-example. Let A_i be the random variable representing the arrival time at location i and M_i represent the service time at location i . Let Ω_{it} be the random variables representing the wait time at location i . Let R_i be the reward collected at location i . In Table 8, the first column presents the two arrival times considered at location 1. The second column reports the wait time distribution with respect to each arrival time and the given queue length of 2. We use “>” to denote the situations in which the traveler will not be able to collect reward from the location even if she waits till the end of the time window. We assume the travel time between the two locations to be 20 and the service time at location 1 is 10. The third and fourth columns report whether the traveler can collect reward from location 1. The fifth through the seventh columns correspond to the service time at location 1, the arrival time at location 2, and the expected reward from location 2 if the traveler stays in queue at location 1. The ninth and tenth columns present the arrival time at location 2 and the expected rewards from location 2 if balking at location 1. Note that the expected rewards from location 2 in the seventh and tenth columns are computed in Table 9. The eighth and eleventh columns present the overall expected rewards from the two locations for the action of staying in queue and balking at location 1, respectively. Similarly, in Table 9, the first column includes the arrival times reported in the sixth and ninth columns of Table 8. The second and third columns present the queue length and wait time distributions, respectively. The fourth column indicates whether the traveler can collect reward from location 2 and the fifth column report the reward collected at location 2. The sixth column presents the expected reward from location 2 according to the queue length and wait time distributions.

When the traveler arrives at location 1 at the time of 10, with location 2 having greater value, by leaving location 1 earlier she may be more likely to collect reward from location 2 and thereby maximize the total expected reward from the tour. If arriving at location 1 at the later time of 25, however, the traveler may be less likely to collect reward from location 2, even if leaving location 1 immediately upon arrival. Thus, it is better for her to stay in line at location 1. As shown in Table 8, for an arrival time of 10, it is optimal to balk and go to location 2 because $E[\text{Stay at 1}] = 30 < E[\text{Leave 1}] = 34$. For an arrival time of 25, with $E[\text{Stay at 1}] = 6 > E[\text{Leave 1}] = 2$, it is

Arrive Time	Wait Time Dist.	Stay at Location 1						Leave Location 1		
A_1	$\omega_{1t}, P(\omega_{1t})$	Collect Reward	R_1	M_1	A_2	$E[R_2]$	$E[\text{Stay at 1}]$	A_2	$E[R_2]$	$E[\text{Balk at 1}]$
10	(10, 0.4)	Yes	30	10	50	0	30	30	34	34
	(20, 0.6)	Yes	30	10	60	0				
25	(10, 0.2)	Yes	30	10	65	0	6	45	2	2
	(>15, 0.8)	No	0	0	60	0				

Table 8: Location 1 with time window $[0, 40]$

Arrive Time	Queue Dist.	Wait Time Dist.	Collect Reward	R_2	$E[R_2]$
A_2	$q_{2t}, P(q_{2t})$	$\omega_{2t}, P(\omega_{2t})$			
30	(1, 0.6)	(10, 0.3)	Yes	50	34
	(1, 0.6)	(15, 0.7)	Yes	50	
	(2, 0.4)	(20, 0.2)	Yes	50	
	(2, 0.4)	(>30, 0.8)	No	0	
45	(2, 0.4)	(10, 0.1)	Yes	50	2
	(2, 0.4)	(>15, 0.9)	No	0	
	(3, 0.6)	(>15, 1)	No	0	
50	(3, 0.2)	(>10, 1)	No	0	0
	(4, 0.8)	(>10, 1)	No	0	

Table 9: Location 2 with time window $[0, 60]$

optimal to stay in queue at location 1.

In the following, we present the proofs to Theorem 1 and Theorem 2, which investigate the conditions under which suboptimal actions can be eliminated.

Theorem 1. *If the traveler is in a queue of length $q > 0$ at location i at time $t < o_j - c_{ij} - \hat{m}_i - 1$, then the action of leaving location i to go to location j can be eliminated at time t .*

Proof. While being at location i at time $t < o_j - c_{ij} - \hat{m}_i - 1$ and observing queue length $q > 0$, the traveler can either leave location i and go to location j or stay at location i . If going to location j , the traveler will arrive at $t + c_{ij} < o_j$. In this case, the earliest time at which the traveler could earn a reward is o_j . If staying at location i , the traveler will observe a queue length of q' at time $t + 1$. The value of q' depends on whether there is a queueing event and what it is at time $t + 1$. If $q' > 0$, the traveler can leave i and arrive at j at time $t + 1 + c_{ij} < o_j$. If $q' = 0$, the traveler will enter service at location i and be able to arrive at j at time $t + 1 + c_{ij} + \hat{m}_i < o_j$. Because no competitors will start arriving at location j before time o_j , no matter which one of the above three scenarios occur, the traveler can join the same sample path at location j as she would have had left location i at time t . Meanwhile, compared with heading to location j at time t , the traveler may

have the chance to enter service by staying at location i at time t . Thus, there is no value in the traveler leaving location i and going to location j at time t . $\square$

Theorem 2. *Assuming that the minimum service time is one time unit and that competitors in the queue will not renege, if the traveler is in a queue of length $q > 0$ at location i at time t and $t + q > l_i$, then the action of staying at location i can be eliminated.*

Proof. Given the time window restriction and the assumption that competitors do not renege, there is no sample path that will result in a queue length $q' = 0$ at time l_i , while observing a queue length of $q > 0$ at time t such that $t + q > l_i$. Therefore, there is no value in the traveler staying at location i at time t . $\square$

D Variable Neighborhood Descent Algorithm

Algorithm 2: Variable Neighborhood Descent (VND)

```

1: Input: A pre- or post-decision state  $s$ , an ordered set of neighborhoods  $N$ 
2: Output: An a priori routing policy  $\pi(\nu)$ 
3:  $\pi(\nu) \leftarrow \text{initialize}()$ 
4:  $iter \leftarrow 0, k \leftarrow 1, level \leftarrow 1, count \leftarrow 1, improving \leftarrow true$ 
5: while runtime < 59 seconds and  $iter < iterationMax$  or  $level < levelMax$  do
6:    $\pi'(\nu) \leftarrow Shake(\pi(\nu))$ 
7:   while  $improving$  and runtime < 59 seconds do
8:      $\pi''(\nu) \leftarrow BestNeighbor(\pi'(\nu), k)$ 
9:     if  $V^{\pi''(\nu)}(s) > V^{\pi'(\nu)}(s)$  then
10:       $\pi'(\nu) \leftarrow \pi''(\nu), count \leftarrow 1$ 
11:     else if  $k = |N|$  and  $count < |N|$  then
12:       $k \leftarrow 1, count \leftarrow count + 1$ 
13:     else if  $k < |N|$  and  $count < |N|$  then
14:       $k \leftarrow k + 1, count \leftarrow count + 1$ 
15:     else
16:       $improving \leftarrow false$ 
17:   if  $V^{\pi''(\nu)}(s) > V^{\pi(\nu)}(s)$  then
18:      $\pi(\nu) \leftarrow \pi''(\nu)$ 
19:      $level \leftarrow 1$ 
20:   else
21:      $level \leftarrow level + 1$ 
22:    $iter \leftarrow iter + 1$ 

```

We present the variable neighborhood descent (VND) heuristic used to search the a-priori-route policy for approximating the reward-to-go in Algorithm 2. Line 3 of Algorithm 2 initializes the search with an a-priori-route policy from state s with randomly ordered locations. Line 5 states the termination criteria for the VND. As mentioned in §5.2, the minimum time increment in our

discrete Markov process is one minute, so we restrict the heuristic search time for an a-priori-route policy to 59 seconds in Line 5. Within the time limit, we use the variable *iter* and *level* to ensure that the algorithm performs at least *iterationMax* iterations in total and *levelMax* iterations after finding an improved solution. We determine that setting *iterationMax* = 35 and *levelMax* = 15 balances the tradeoff between computational efficiency and heuristic performance. The *Shake* procedure in Line 6 randomly re-sequences locations in policy $\pi(\nu)$. Line 7 through Line 16 find a locally-optimal policy relative to a set of specified neighborhoods N , which is composed of 1-shift and 2-opt neighborhoods in our study. The *BestNeighbor* function in Line 8 returns the best policy in the neighborhood k of the current policy $\pi'(\nu)$. Line 9 through Line 16 manage the updating of the locally optimal policy, the active neighborhood, and the variable *count* that ensures all neighborhoods are explored before terminating. Line 17 through Line 21 update the current solution and the value of *level*. Line 22 advances the interaction counters.

References

- Bertsekas, D. P. 2005. *Dynamic Programming and Optimal Control*, vol. I. 3rd ed. Athena Scientific, Belmont, MA.
- Bertsekas, D. P., J. H. Tsitsiklis, C. Wu. 1997. Rollout algorithms for combinatorial optimization. *Journal of Heuristics* **3**(3) 245–262.
- Burnetas, A. N. 2013. Customer equilibrium and optimal strategies in Markovian queues in series. *Annals of Operations Research* **208** 515–529.
- Campbell, A. M., M. Gendreau, B. W. Thomas. 2011. The orienteering problem with stochastic travel and service times. *Annals of Operations Research* **186** 61–81.
- Campbell, A. M., B. W. Thomas. 2008. Challenges and advances in a priori routing. B. Golden, S. Raghavan, E. Wasil, eds., *The vehicle routing problem: latest advances and new challenges*, *Operations Research/Computer Science Interfaces Series*, vol. 43. Springer, New York, 123–142.
- Chang, H. S., M. C. Fu, J. Hu, S. I. Marcus. 2013. Simulation-based algorithms for Markov decision

- processes. *Communications and Control Engineering*, 2nd ed., chap. 5. Springer, London, 179–226.
- Cheng, S., X. Qu. 2009. A service choice model for optimizing taxi service delivery. *Proceedings of the 12th International IEEE Conference on Intelligent Transportation Systems*. IEEE, Piscataway, NJ, 66–71.
- Evers, L., K. Glorie, S. van der Ster, A. I. Barros, H. Monsuur. 2014. A two-stage approach to the orienteering problem with stochastic weights. *Computers & Operations Research* **43** 248–260.
- Feillet, D., P. Dejax, M. Gendreau, C. Gueguen. 2004. An exact algorithm for the elementary shortest path problem with resource constraints: Application to some vehicle routing problems. *Networks* **44**(3) 216–229.
- Goodson, J. C., J. W. Ohlmann, B. W. Thomas. 2013. Rollout policies for dynamic solutions to the multivehicle routing problem with stochastic demand and duration limits. *Operations Research* **61**(1) 138–154.
- Goodson, J. C., B. W. Thomas, J. W. Ohlmann. 2015. Restocking-based rollout policies for the vehicle routing problem with stochastic demand and duration limits. *Transportation Science* **50**(2) 591–607.
- Goodson, J. C., B. W. Thomas, J. W. Ohlmann. 2016. A generalized rollout policy framework for stochastic dynamic programming. <http://www.slu.edu/~goodson/papers/GoodsonRolloutFramework.pdf>. Working paper.
- Gupta, Anupam, Ravishankar Krishnaswamy, Viswanath Nagarajan, R Ravi. 2014. Running errands in time: Approximation algorithms for stochastic orienteering. *Mathematics of Operations Research* **40**(1) 56–79.
- Honnappa, H., R. Jain. 2015. Strategic arrivals into queueing networks: The network concert queueing game. *Operations Research Articles in Advance*.
- Kim, Y., H. Ko, S. Pack, W. Lee, X. Shen. 2013. Mobility-aware call admission control algorithm with handoff queue in mobile hotspots. *IEEE Transactions on Vehicular Technology* **62**(8) 3903–3912.

- Klapp, Mathias A., Alan L. Erera, Alejandro Toriello. to appear. The one-dimensional dynamic dispatch waves problem. *Transportation Science* .
- Manfredi, S. 2014. Decentralized queue balancing and differentiated service scheme based on cooperative control concept. *IEEE Transactions on Industrial Informatics* **10**(1) 586–593.
- Novoa, C., R. Storer. 2009. An approximate dynamic programming approach for the vehicle routing problem with stochastic demands. *European Journal of Operational Research* **196**(2) 509–515.
- Papapanagiotou, V., D. Weyland, R. Montemanni, L. M. Gambardella. 2013. A sampling-based approximation of the objective function of the orienteering problem with stochastic travel and service times. *Lecture Notes in Management Science* **5** 143–152.
- Powell, W.B. 2007. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*, Wiley Series in Probability and Statistics, vol. 703. Wiley-Interscience, Hoboken, New Jersey.
- Scholtes, S. 2001. Markov chain. <http://www.eng.cam.ac.uk/~ss248/G12-M01/Week3/Lecture.ppt>.
- Secomandi, N. 2000. Comparing neuro-dynamic programming algorithms for the vehicle routing problem with stochastic demands. *Computers & Operations Research* **27** 1201–1224.
- Secomandi, N. 2001. A rollout policy for the vehicle routing problem with stochastic demand. *Operations Research* **49**(5) 796–802.
- Solomon, M. M. 1987. Algorithms for the vehicle routing and scheduling problem with time window constraints. *Operations Research* **35** 254–265.
- Tang, H., E. Miller-Hooks. 2005. Algorithms for a stochastic selective travelling salesperson problem. *Journal of the Operational Research Society* **56**(4) 439–452.
- Teng, S. Y., H. L. Ong, H. C. Huang. 2004. An integer L-shaped algorithm for time-constrained traveling salesman problem. *Asia-Pacific Journal of Operational Research* **21**(2) 241–257.
- Toriello, A., W. B. Haskell, M. Poremba. 2014. A dynamic traveling salesman problem with stochastic arc costs. *Operations Research* **62**(5) 1107–1125.

- Ulmer, Marlin W., Justin C. Goodson, Dirk C. Mattfeld, Marco Hennig. 2015a. Offline-online approximate dynamic programming for dynamic vehicle routing with stochastic requests. Submitted for publication.
- Ulmer, Marlin W., Dirk C. Mattfeld, Marco Hennig, Justin C. Goodson. 2015b. A rollout algorithm for vehicle routing with stochastic customer requests. *Logistics Management*. Springer, 217–227.
- Vansteenwegen, P., W. Souffriau. 2010. Tourist trip planning functionalities: State-of-the-art and future. *Lecture Notes in Computer Science* **6385** 474–485.
- Vansteenwegen, P., W. Souffriau, D. V. Oudheusden. 2011. The orienteering problem: A survey. *European Journal of Operational Research* **209** 1–10.
- Voccia, S., A. M. Campbell, B. W. Thomas. 2013. Probabilistic traveling salesman problem with time windows. *EURO Journal on Transportation and Logistics* **2** 89–107.
- Yaghmaee, M. H., N. F. Bahalgardi, D. Adjero. 2013. A prioritization based congestion control protocol for healthcare monitoring application in wireless sensor networks. *Wireless Personal Communications* **72** 2605–2631.
- Yechiali, U. 1971. On optimal balking rules and toll charges in the GI/M/1 queuing process. *Operations Research* **19**(2) 349–371.
- Yechiali, U. 1972. Customers' optimal joining rules for the G1/M/s queue. *Management Science* **18**(7) 434–443.
- Zhang, S., J. W. Ohlmann, B. W. Thomas. 2014. A priori orienteering with time windows and stochastic wait times at customers. *European Journal of Operational Research* **239** 70–79.